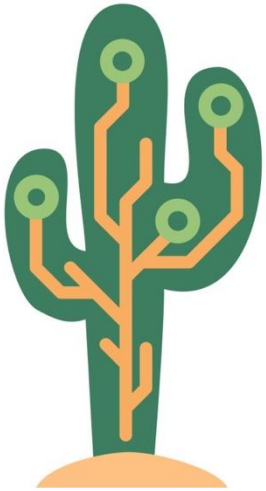
Ira A. Fulton Schools of Engineering
Arizona State University

Advanced Hardware and Systems for Machine Learning

Lecture 7, 8, 9: Hardware Metrics

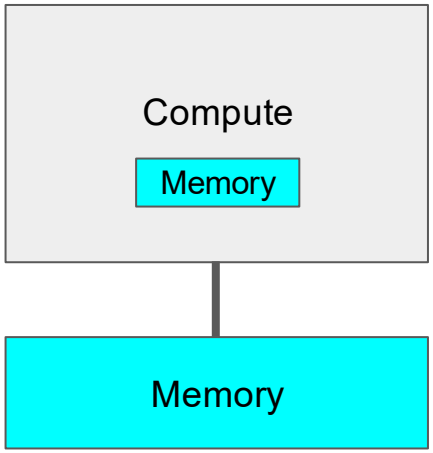


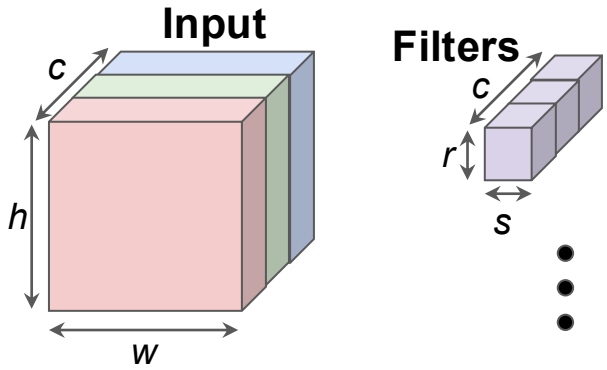
Jeff (Jun) Zhang
Assistant Professor
School of ECEE, ASU
jeffzhang@asu.edu



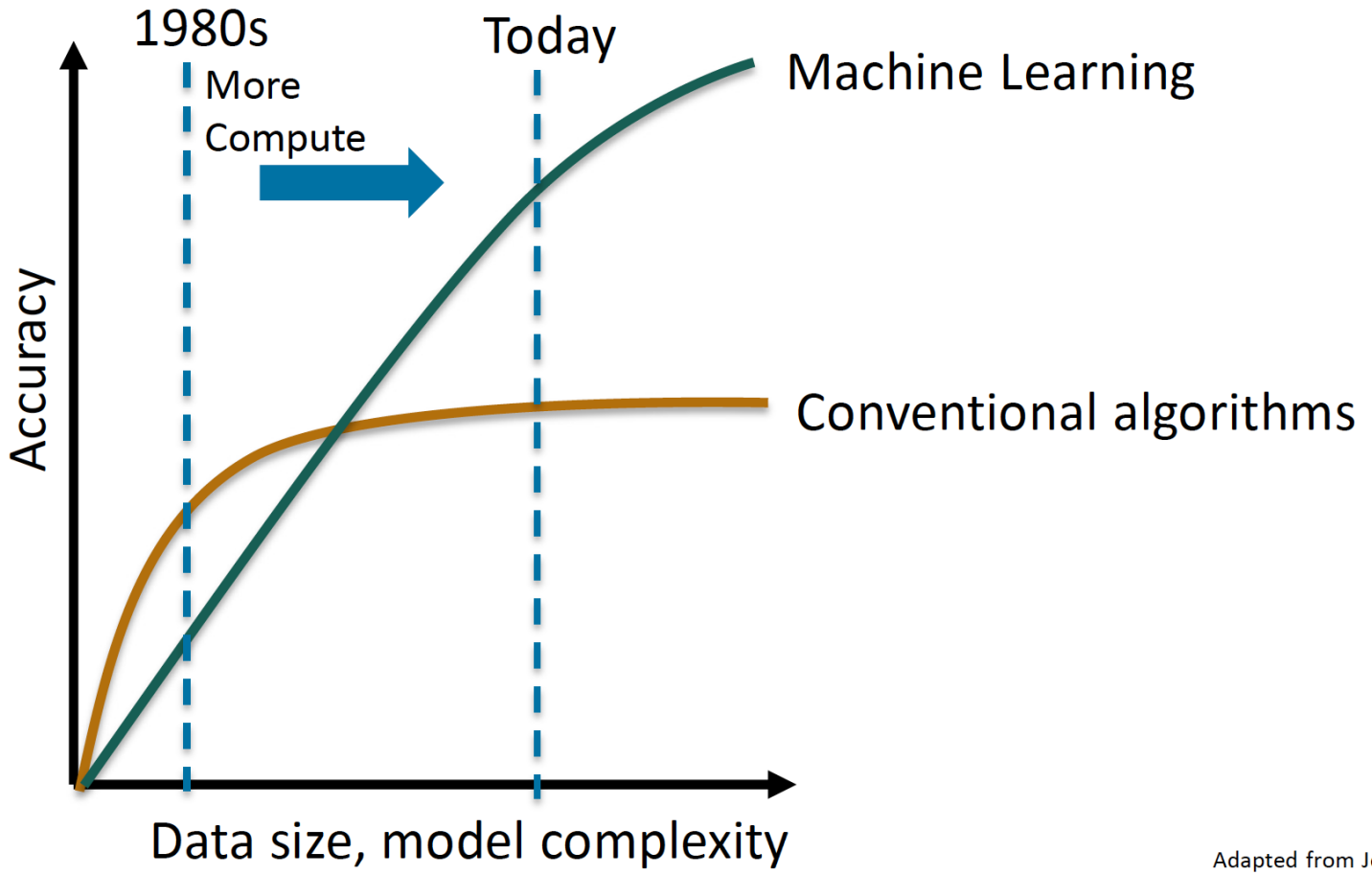
Recap

- Von-Neumann architecture (and bottleneck)
- Overlap mem access and computation
- Memory vs. compute of important DNN components
  - Convolutions
  - Fully-connected/ Linear
  - Attention
  - Bottleneck block
  - LSTM



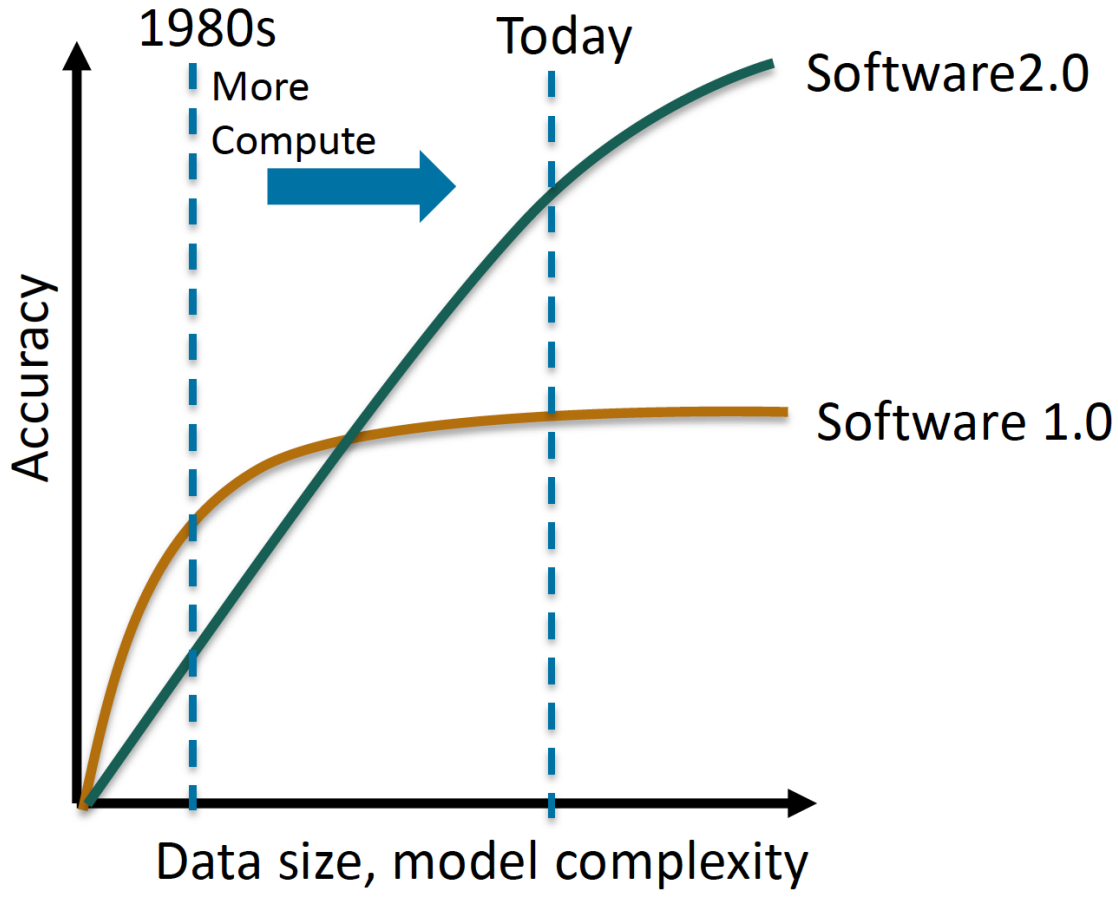


Machine Learning Today



Adapted from Jeff Dean
HotChips 2017

Machine Learning Today



Adapted from Jeff Dean
HotChips 2017

Software 1.0 vs Software 2.0

Software 1.0

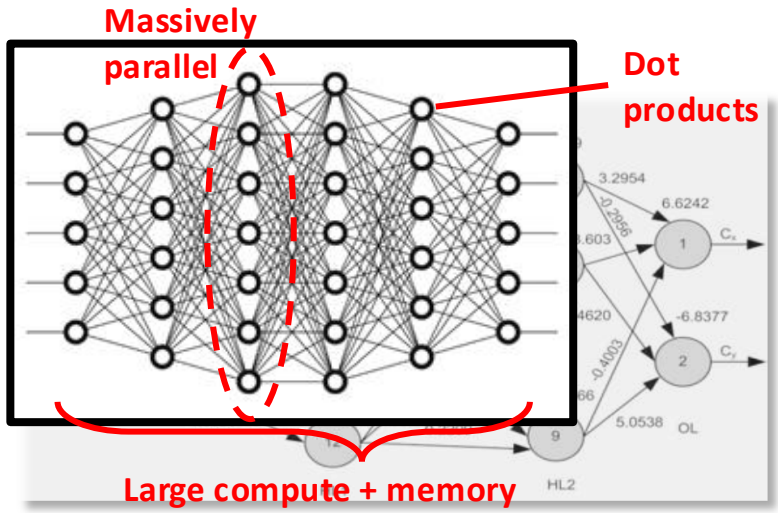
Sequential

Control flow

```
template<typename InputIterator1, typename InputIterator2, typename NumericT>
NumericT  inner_product(InputIterator1 start1,
                        InputIterator1 end1,
                        InputIterator2 start2,
                        NumericT       startval) const {
  for (; start1 != end1; ++start1, ++start2)
    startval += ( (*start1) * (*start2) );
  return startval;
}
// ..
vector_type vec1, vec2;
vector_type::numeric_t val;
val = inner_product(vec1.begin(), vec1.end(), vec2.begin(),
                    vector_type::numeric_t(0));
```

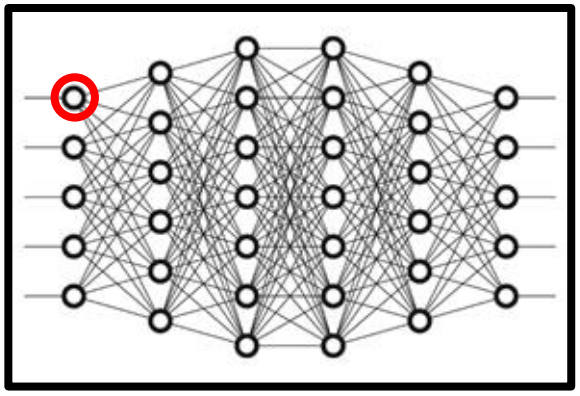
- Written in code (C++, …)
- Requires domain expertise
  - Decompose the problem
  - Design algorithms
  - Compose into a system

Software 2.0



- Programmer input: training data
- Written in the weights of a neural network model by optimization

Hardware *Enables* Deep Learning

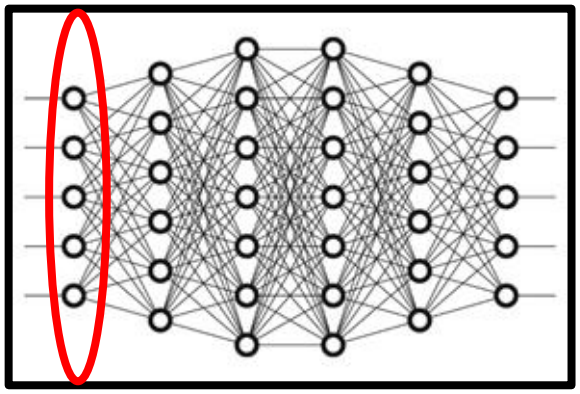




Sequential
Complex instruction set
Great for control flow

- 1986: Backpropagation published
- ~30 years of trying this on CPUs

Hardware *Enables* Deep Learning





Parallel
Great for matrix math
Bad for control flow

- 1986: Backpropagation published
- ~30 years of trying this on CPUs
- 2012: AlexNet paper ⇨ +10% accuracy ⇨ Deep learning explosion

Accelerators for ML



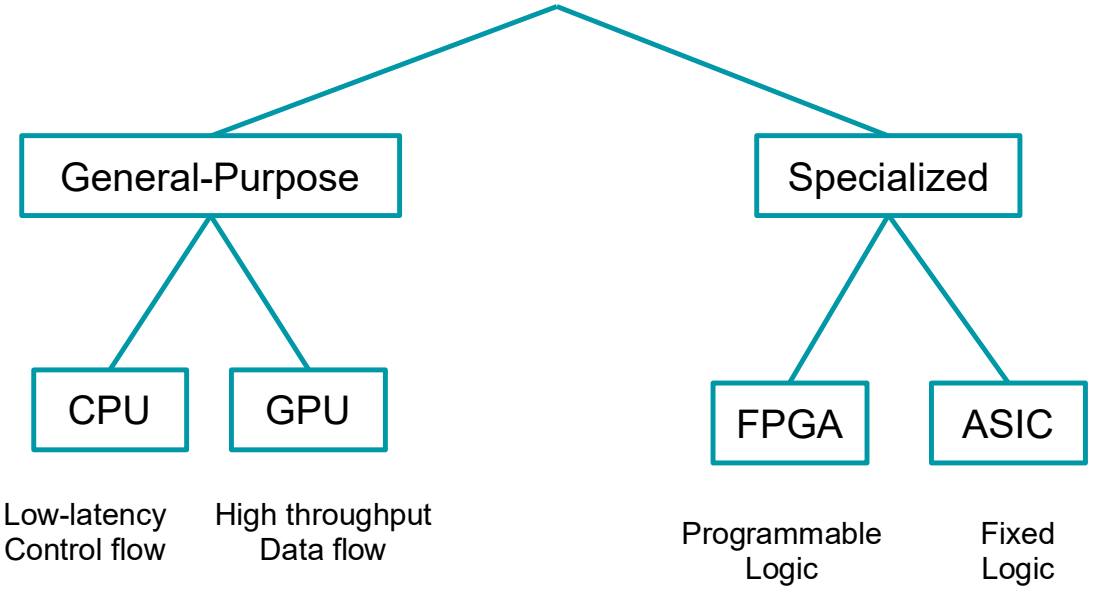








CPU	GPU	FPGA	TPU	Next
Threads SIMD	Massive Threads SIMD HBM	LUTs DSP BRAM	MM Unit BRAM	???



[Standford CS217]

DNNs are Getting *Much* Bigger



AlexNet

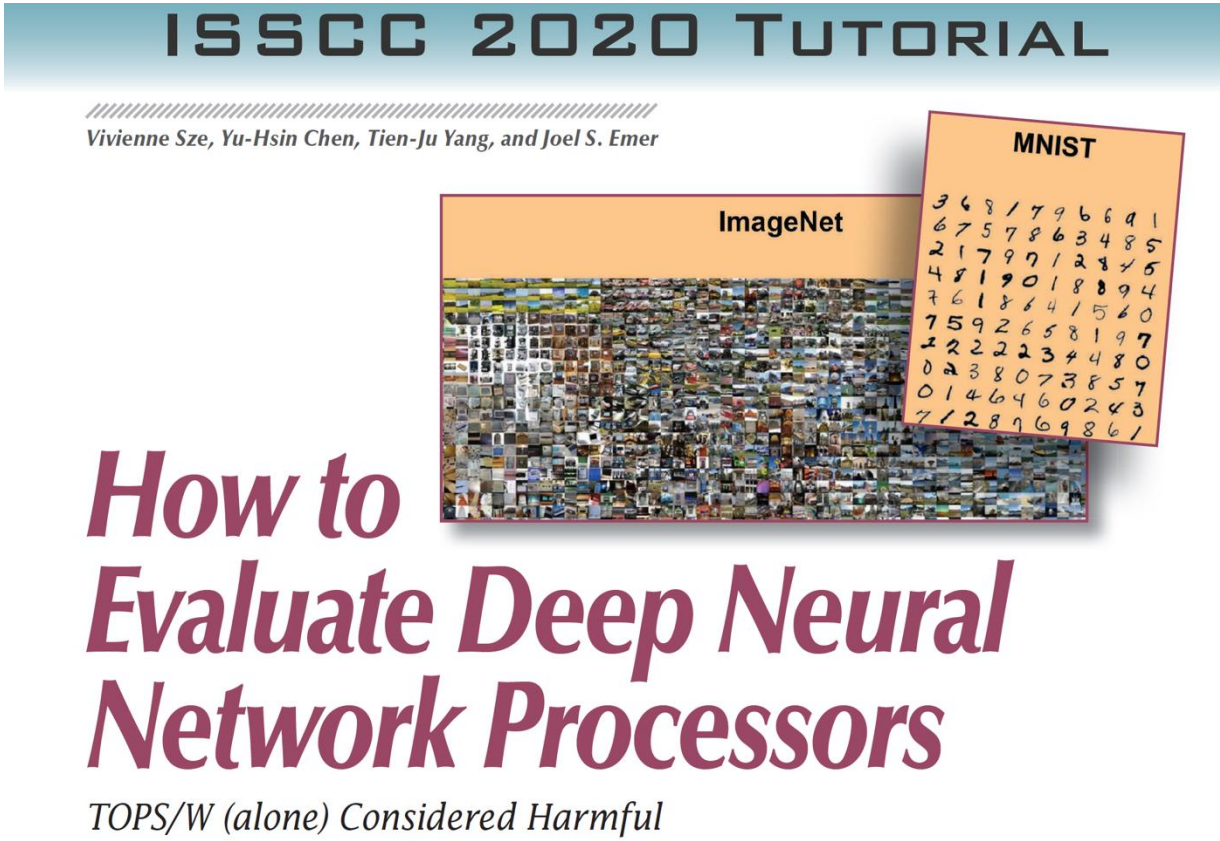
- 10 years old
- 60M parameters
- Data: 1M images
- 12 GPU days

GPT-3

- SOTA language model
  - Data: 10B ↦ 500B tokens
- 355 GPU years to train
  - 175 Billion parameters
  - $3x10^{23}$ FLOPS to train
- $4.6M USD *(AWS Estimate)*
- Carbon footprint:
  - 176 house years
  - 700,000 km in a car

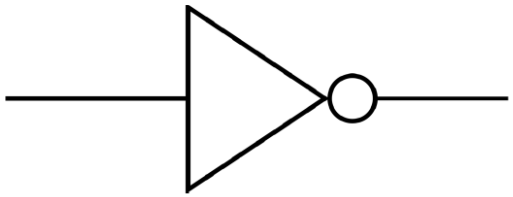
Key Metrics and Design Objectives

How to build *efficient* DNN Hardware?



GOPS/W or TOPS/W

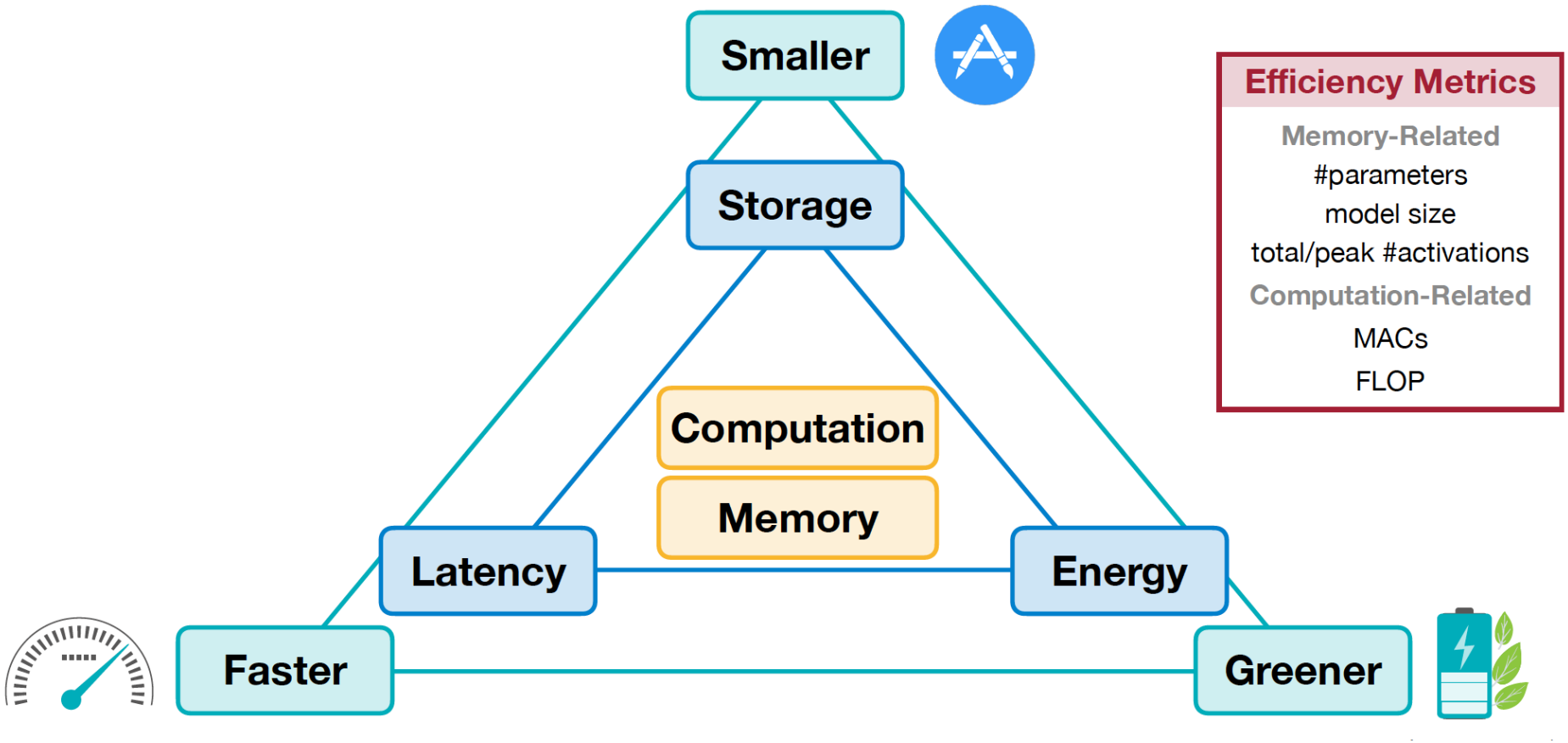
- GOPS = giga (10^9) operations per second
- TOPS = tera (10^{12}) operations per second
- GOPS/Watt or TOPS/Watt commonly reported in hardware literature to show **efficiency** of design
- However, does not provide sufficient insights on hardware capabilities and limitations (especially if based on peak throughput/performance)



Example: high TOPS per watt can be achieved with inverter (ring oscillator)

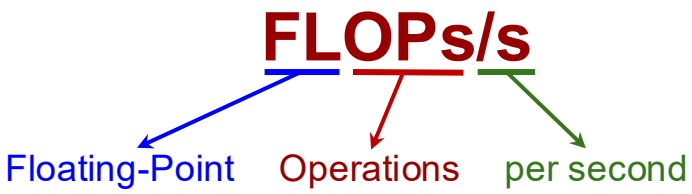
[Sze and Emer]

Efficiency Metrics for DNNs

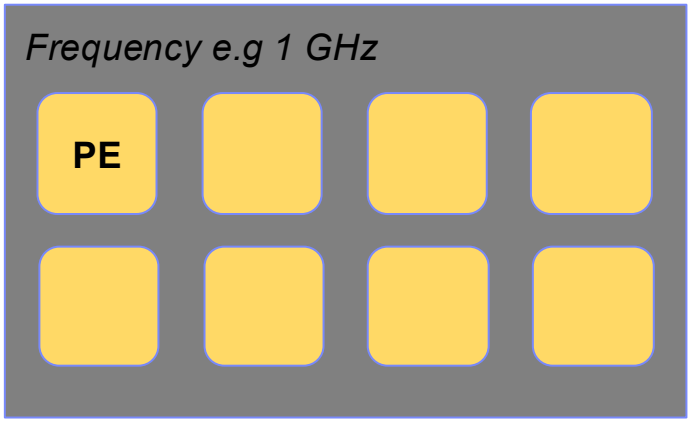


[Han]

Compute Performance Metrics



- MACs/s: Multiply-accumulate Ops/s
  - For many DNN hardware
- OPs/s: for non floating-point operations
- Chips are often labeled with “peak FLOPs/s”
  - Not achievable under normal workloads
  - Very rough indication of performance

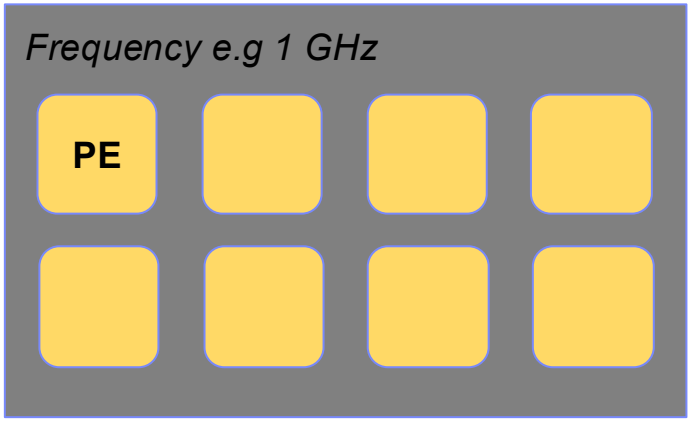


$$\frac{\text{operations}}{\text{second}} = \underbrace{\left(\frac{1}{\frac{\text{cycles}}{\text{operation}}} \times \frac{\text{cycles}}{\text{second}}\right)}_{\text{for a single PE}} \times \text{number of PEs}$$

Compute Performance Metrics

Hardware **FLOPs/s**

Floating-Point Operations per second



DNN Models

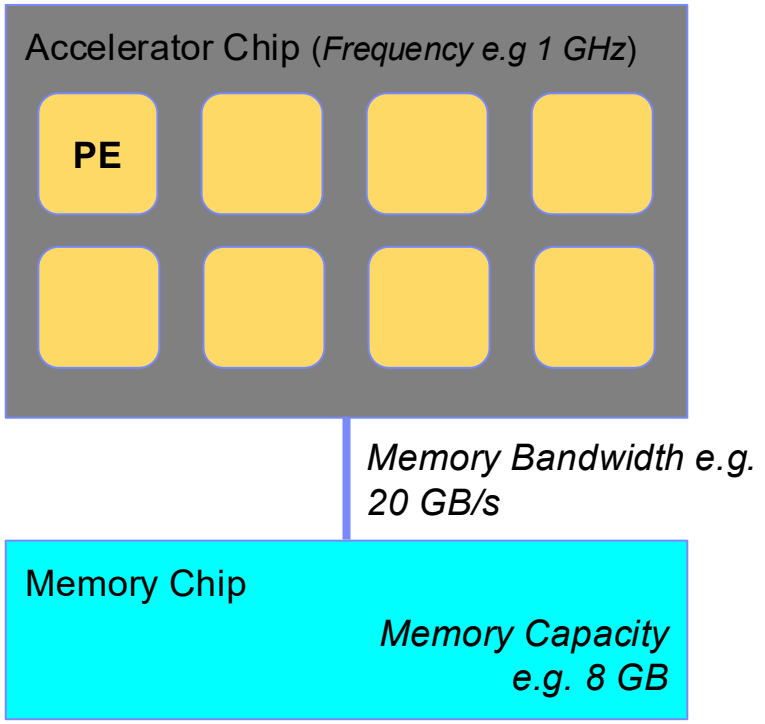
A deep learning model is effectively a bunch of matrix multiplications, each composed of floating-point multiplication and addition 'operations' (FLOPs). Our accelerator speed determines how long these take to compute:

$$T_{\text{math}} = \frac{\text{Computation FLOPs}}{\text{Accelerator FLOPs/s}}$$

For instance, an NVIDIA H100 can perform about 9.89e14 bfloat16 FLOPs/s while a TPU v6e can perform 9.1e14 FLOPs/s.
That means doing 1e12 FLOPs on an H100 will take (roughly) 1e12 / 9.89e14 =1.01ms and 1e12 / 9.1e14 = 1.1ms on a TPU v6e.

Memory Performance Metrics

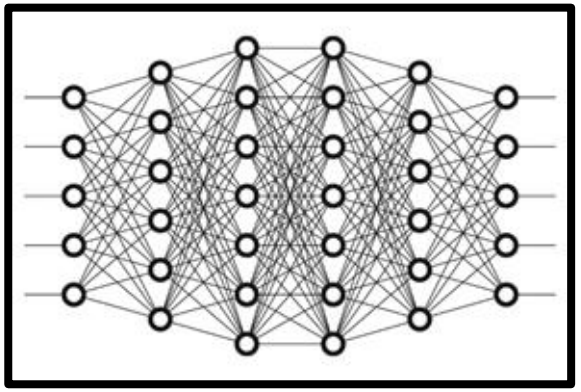
- Memory capacity [GB]
- Memory bandwidth [GB/s]
  - Transfer speed from memory chip to compute chip
- **More complicated** because there is a *memory hierarchy* and *data flow*
  - Showing “external”/”main” memory
  - Can have caches, local memory, registers with much higher bandwidth
  - Data reuse and locality!



Example: “HBM bandwidth”
On an H100, this is about 3.35TB/s and on TPU v6e this is about 1.6TB/s.

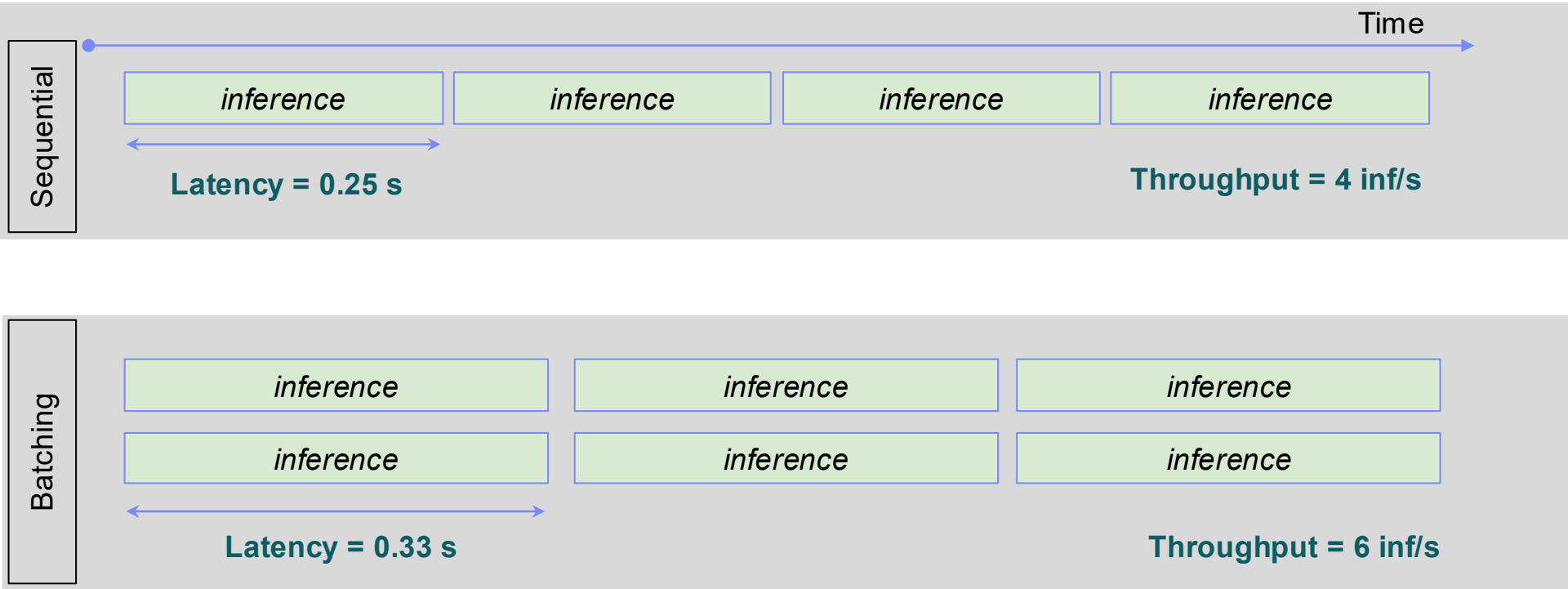
DNN Metrics

- **Compute: FLOPs**
  - Total number of operations needed to run the model
- **Memory Footprint: MBs/GBs**
  - Total size of the model parameters
  - Total size of activations

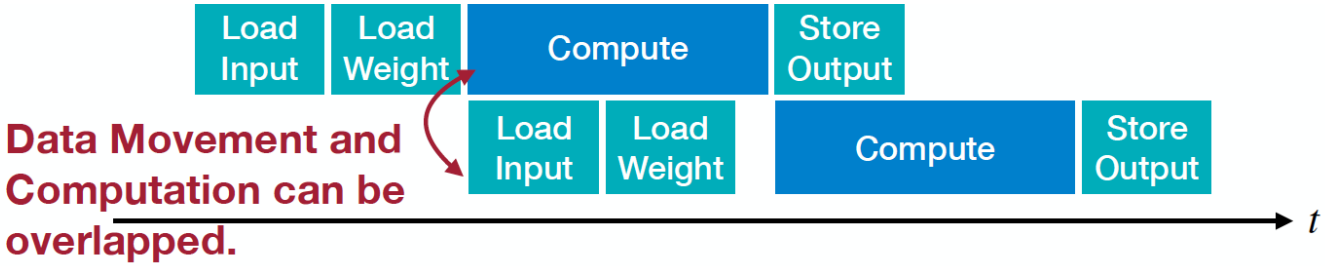


DNN Performance: Throughput and Latency

- **Latency**: Number of seconds per inference (unit = seconds)
- **Throughput**: Number of inferences per second (unit = inference/second)



Latency



$$Latency \approx \max\left(T_{computation}, T_{memory}\right)$$

$$T_{\text{computation}} \approx \frac{\text{Number of Operations in Neural Network Model}}{\text{Number of Operations that Processor can Process Per Second}}$$

NN Specification

Hardware Specification

$$T_{\text{memory}} \approx T_{\text{data movement of activations}} + T_{\text{data movement of weights}}$$

$$T_{\text{data movement of weights}} \approx \frac{\text{Neural Network Model Size}}{\text{Memory Bandwidth of Processor}}$$

NN Specification

Hardware Specification

$$T_{\text{data movement of activations}} \approx \frac{\text{Input Activation Size + Output Activation Size}}{\text{Memory Bandwidth of Processor}}$$

NN Specification

Hardware Specification

[Han]

Roofline Model

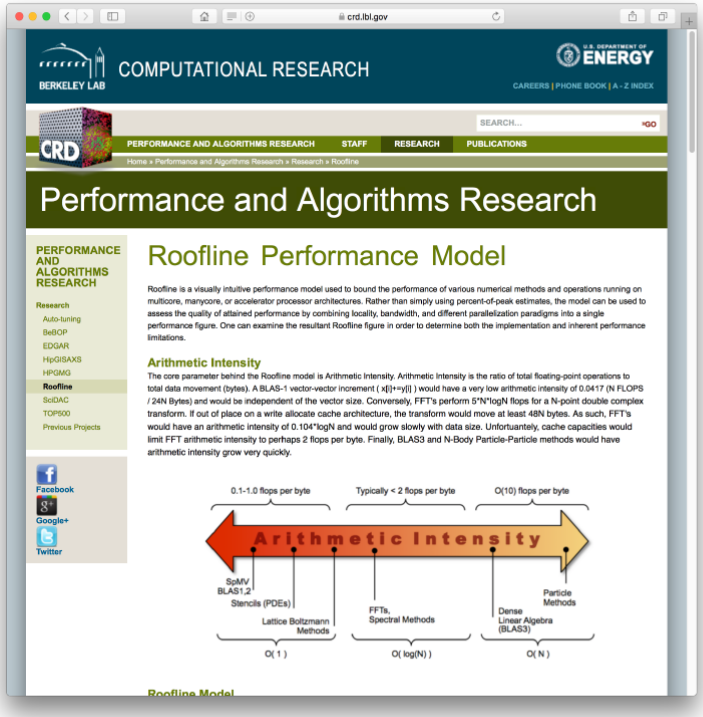
- The **Roofline Model** is a throughput-oriented performance model…
  - Tracks rates not times
  - Augmented with Little's Law

  (concurrency = latency*bandwidth)
  - Independent of ISA and architecture

  (applies to CPUs, GPUs, Google TPUs, etc…)
- Two Components:
  - Machine Characterization

  (realistic performance potential of the system)
  - Theoretical Application Bounds

  (how well could my app perform with perfect compilers, caches, overlap, …)



https://crd.lbl.gov/departments/computer-science/PAR/research/roofline

[Stanford CS217]

Roofline Plot

Characterize the performance of a given hardware device across different workloads

Operational intensity measures the "FLOPs per byte" of a given operation

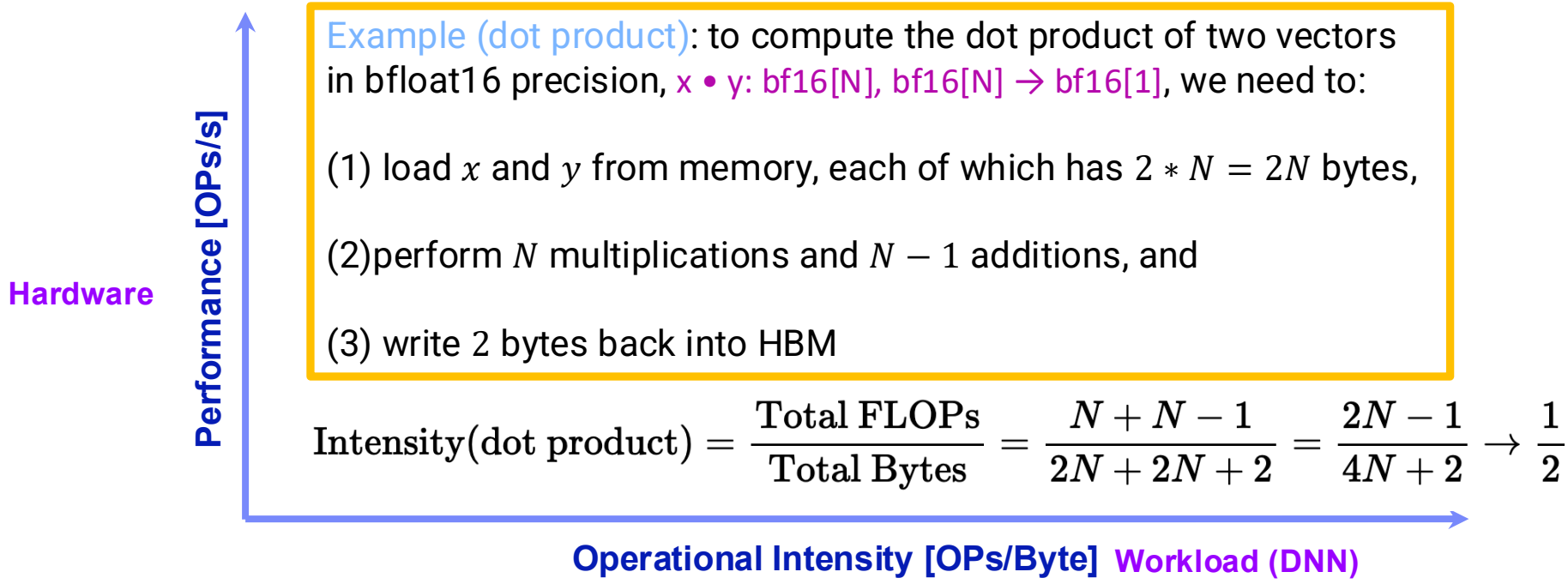
Example (dot product): to compute the dot product of two vectors in bfloat16 precision, x • y: bf16[N], bf16[N] → bf16[1], we need to:

(1) load x and y from memory, each of which has $2 * N = 2N$ bytes,

(2)perform N multiplications and $N - 1$ additions, and

(3) write 2 bytes back into HBM

$$\text{Intensity(dot product)} = \frac{\text{Total FLOPs}}{\text{Total Bytes}} = \frac{N + N - 1}{2N + 2N + 2} = \frac{2N - 1}{4N + 2} \rightarrow \frac{1}{2}$$

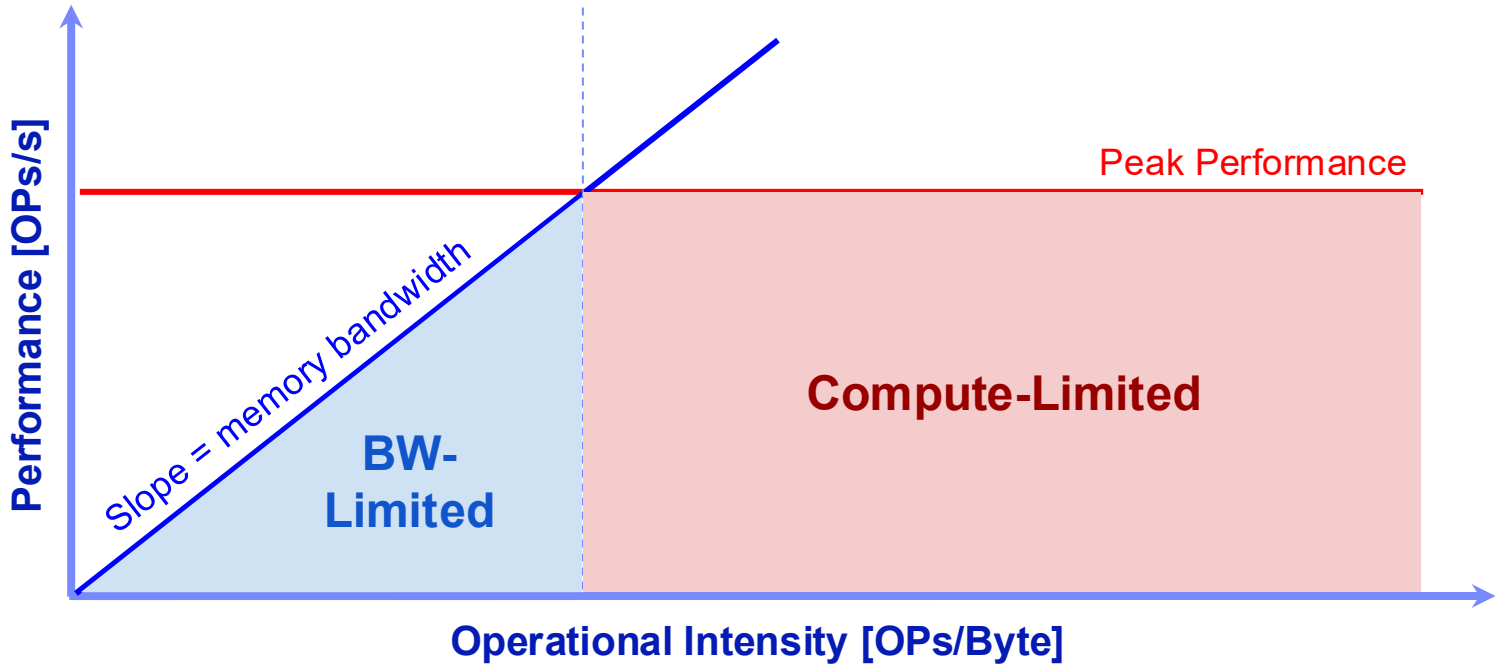


TPU v5e MXU can perform 1.97e14 FLOPs/s and load 8.2e11 bytes/s from HBM. If we always loads its weights from HBM and runs in the MXU:

If an algorithm has a lower arithmetic intensity than 240 FLOPs/byte, it will be bound by byte loading and thus we won't make good use of the hardware.

Roofline Plot

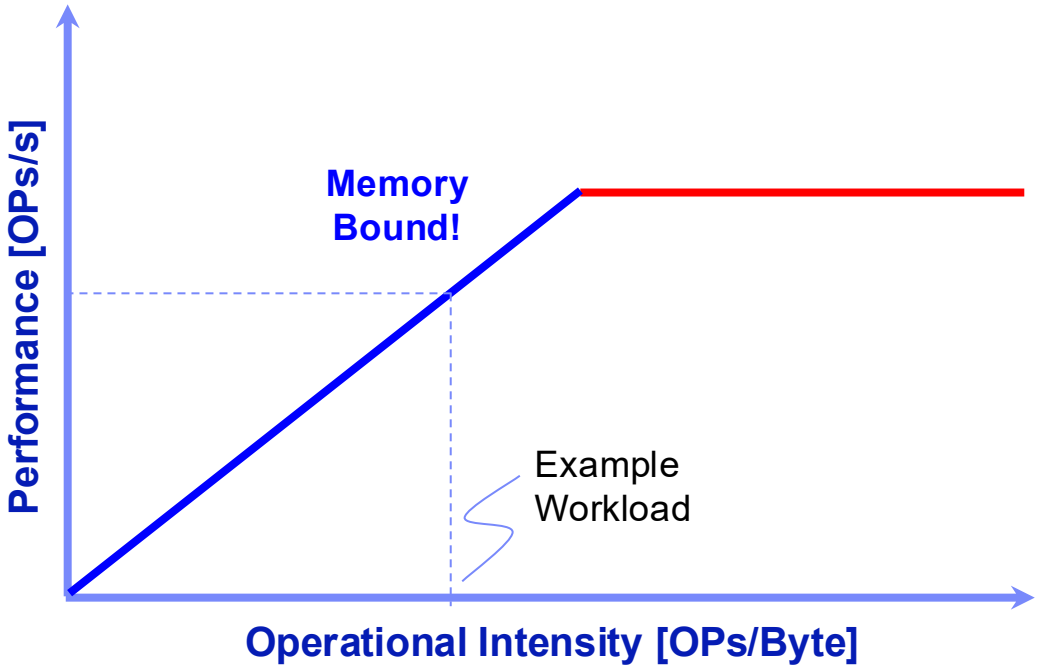
Characterize the performance of a given hardware device across different workloads

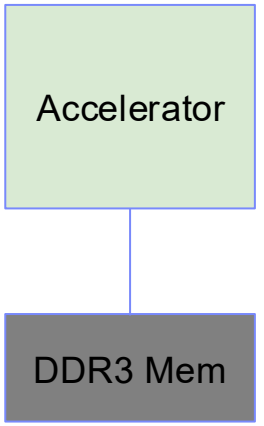


Peak achievable FLOPs/s (throughput) of an algorithm on our hardware (the y-axis) against the arithmetic intensity of that algorithm (the x-axis).

Roofline Plot (DRAM Roofline)

Characterize the performance of a given hardware device across different workloads





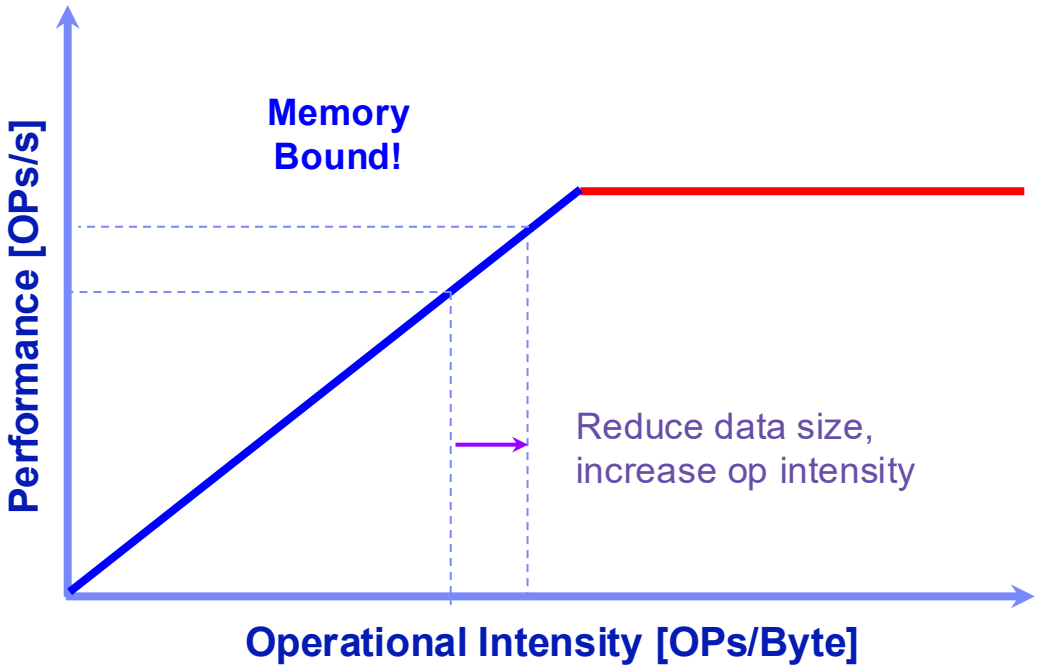
What is OPs/Byte of a DNN?

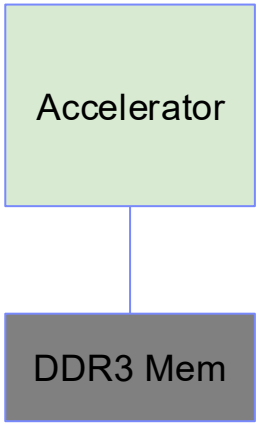
- Operational intensity [OPs/Byte] quantifies the ratio of computations to memory access of a DNN
- Total number of operations = multiplications + additions

$$\text{Operational Intensity} = \frac{\text{Total number of operations}}{\textbf{Bytes accessed}}$$

- **For simplicity we assume,** e.g., a big-enough on-chip mem:
  - Bytes accessed = size of parameters + size of activations
  - Upper bound on the operational Intensity
  - **Pen and Paper Calculation!**

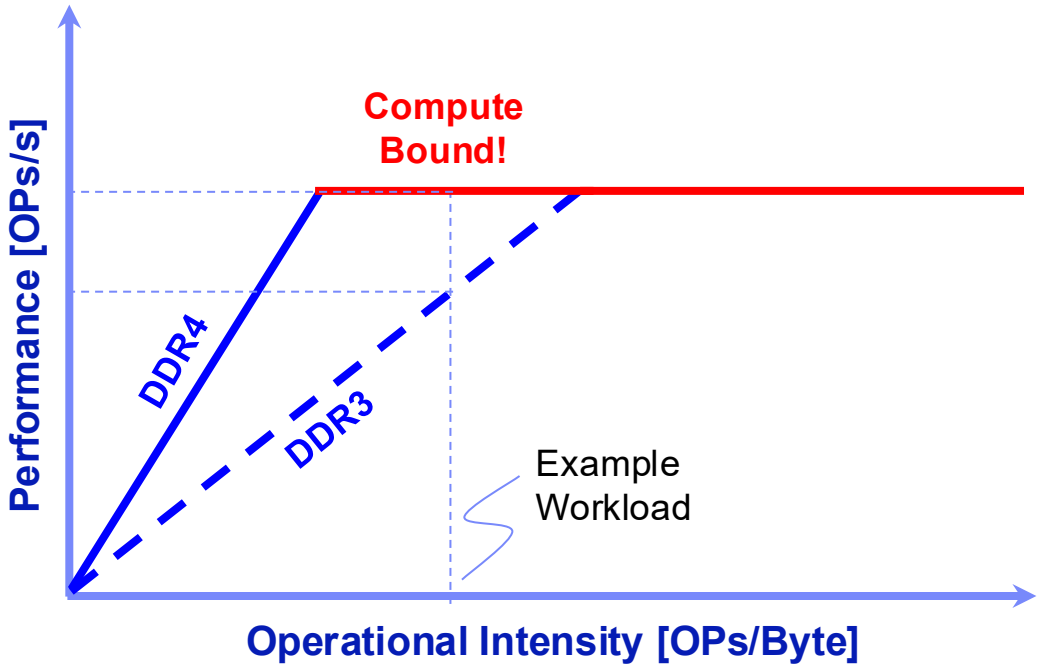
Roofline Plot (DRAM)

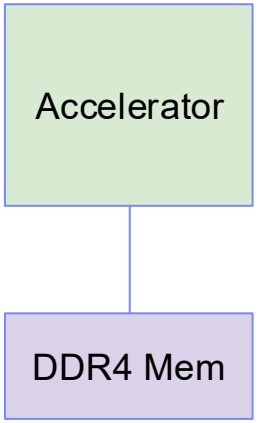




Compressed data format e.g. reduced precision

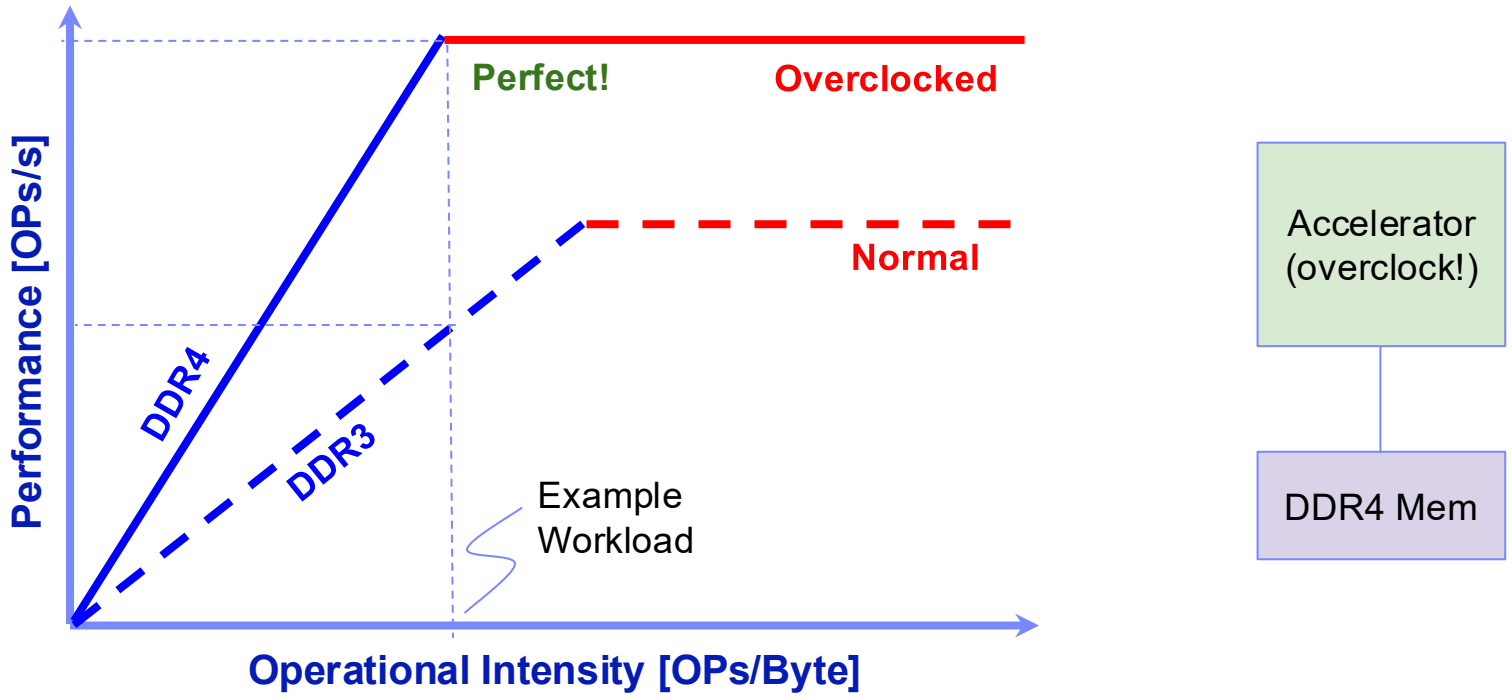
Roofline Plot





Faster memory chip increases slope of roofline

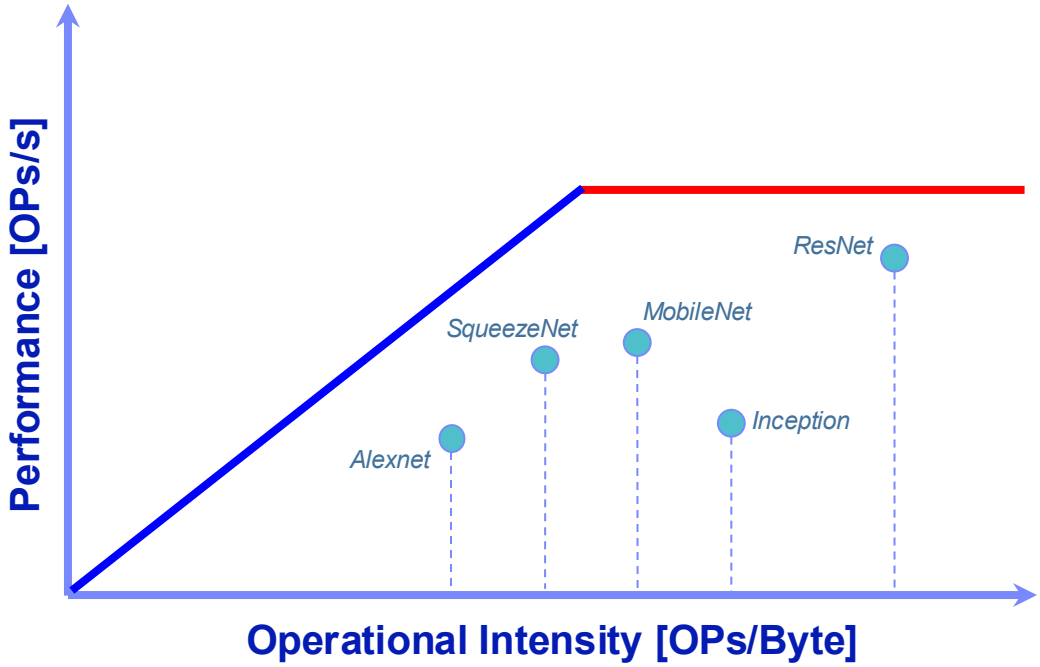
Roofline Plot



Raise the roofline by increasing the speed/throughput of compute

Roofline Example

Measured performance is (by definition) below the roofline.

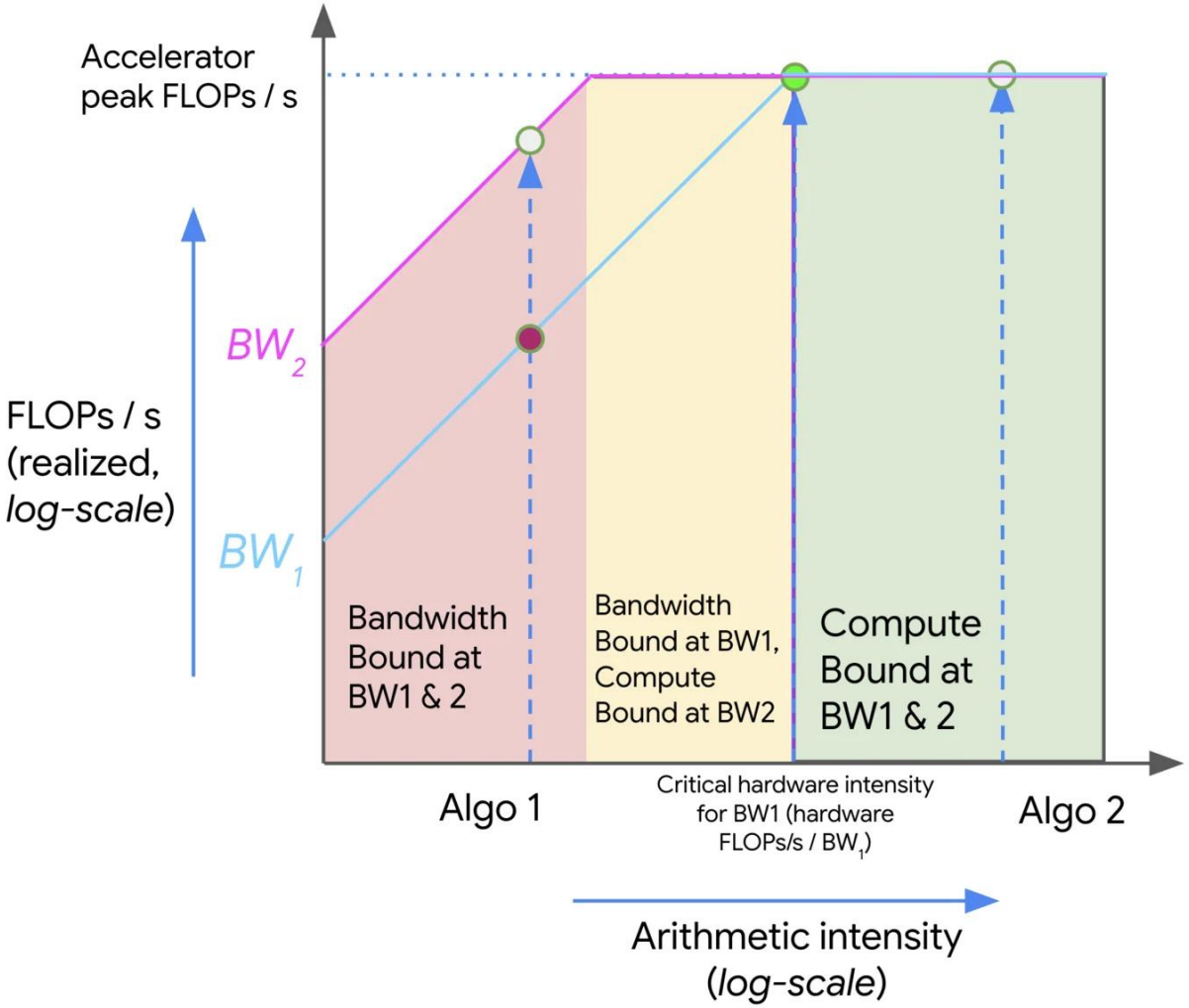


Achieved Performance can be limited by:

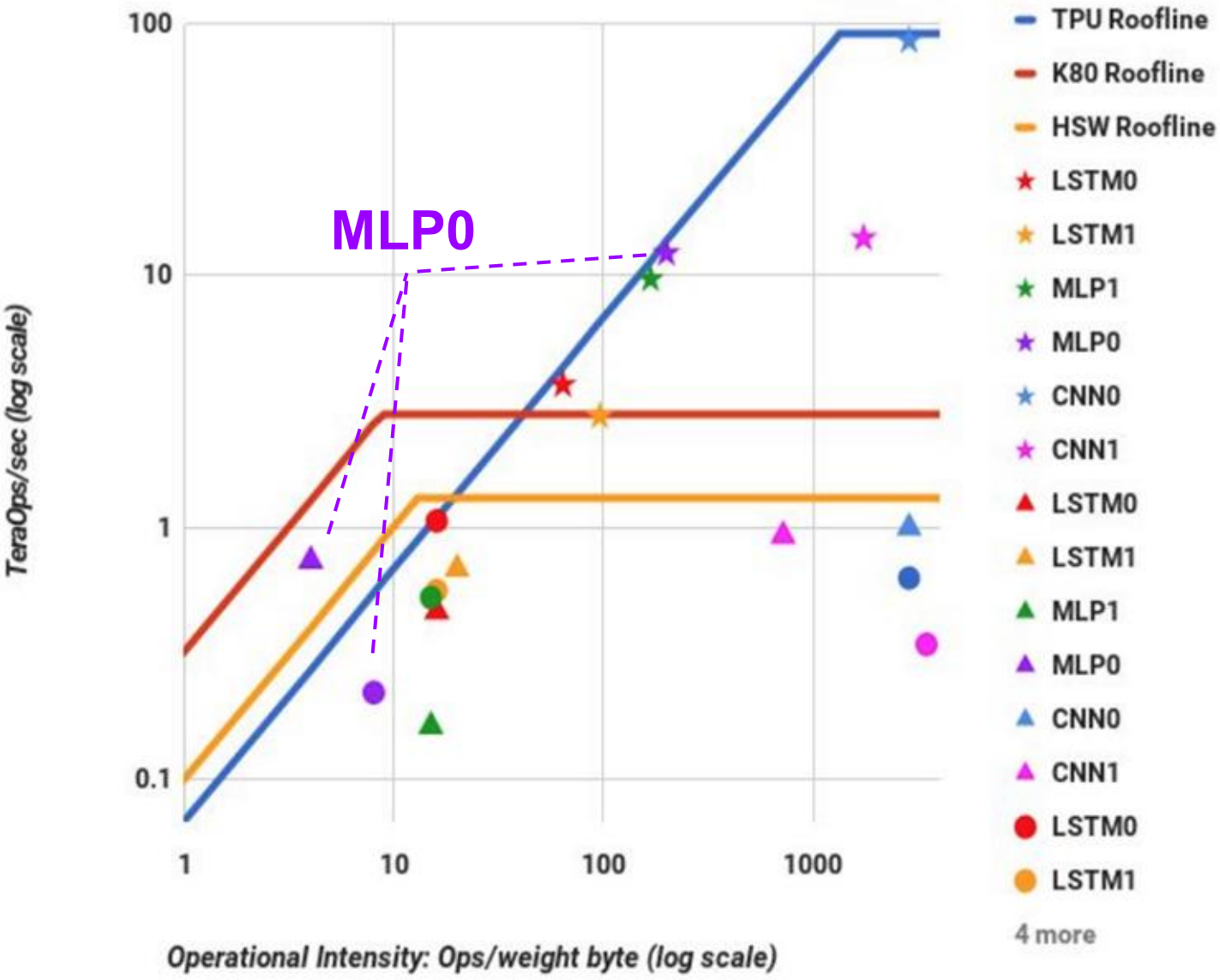
- Memory access efficiency
  - E.g.: uncoalesced reads - most DRAM chips require successive reads, each of a specific width to use maximum bandwidth.
- Compute utilization
  - E.g.: MAC array hardcoded to 16 channels per tile but first layer has 3 channels
  - Overhead of control logic
- Complex DNN topologies
  - Control flow and data hazards may stall execution even if the hardware is available

Note: points are not plotted in their correct place and are just for illustrative purposes

Log-scale


Accelerator
peak FLOPs / s
FLOPs / s
(realized,
log-scale)
BW2
BW1
Bandwidth
Bound at
BW1 & 2
Bandwidth
Bound at BW1,
Compute
Bound at BW2
Compute
Bound at
BW1 & 2
Algo 1
Critical hardware intensity
for BW1 (hardware
FLOPs/s / BW1)
Algo 2
Arithmetic intensity
(log-scale)

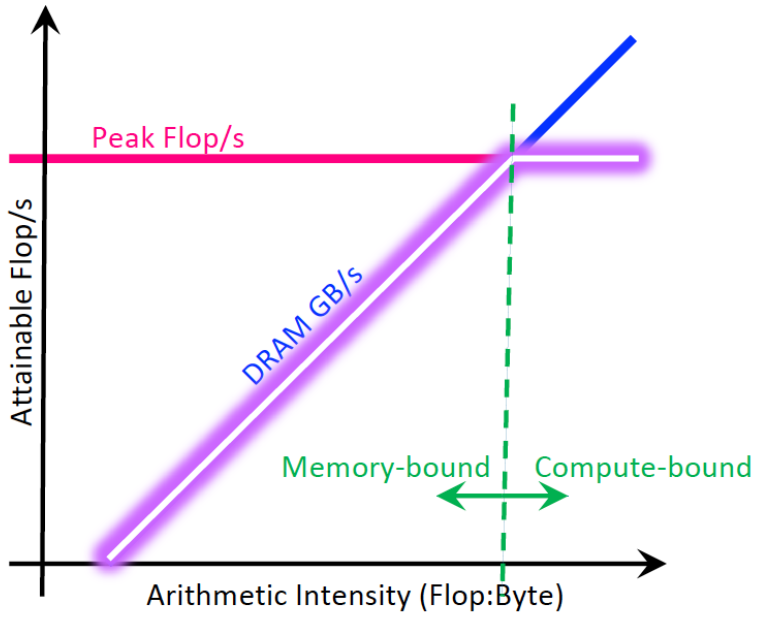
Google TPUv1 Roofline

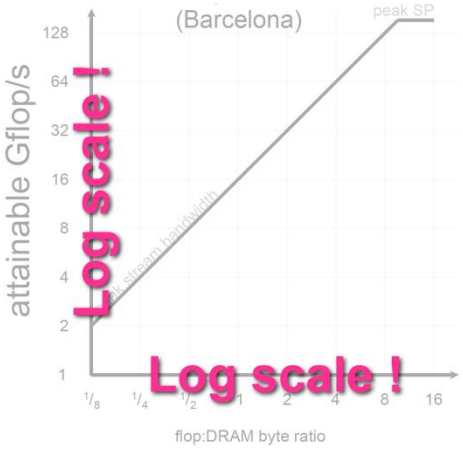


Source: Google

Take a step back: DRAM Roofline

- One could hope to always attain peak performance (Flop/s)
- However, finite locality (reuse) and bandwidth limit performance
- Consider idealized processor/caches
- Plot the performance bound using Arithmetic Intensity (AI) as the x-axis…
  - AI = Flops / Bytes presented to DRAM
  - **Attainable Flop/s = min(peak Flop/s, AI * peak GB/s)**
  - **Log-log scale** makes it easy to doodle, extrapolate performance along Moore's Law, etc…
  - Kernels with AI less than machine balance are ultimately DRAM bound (we'll refine this later…)



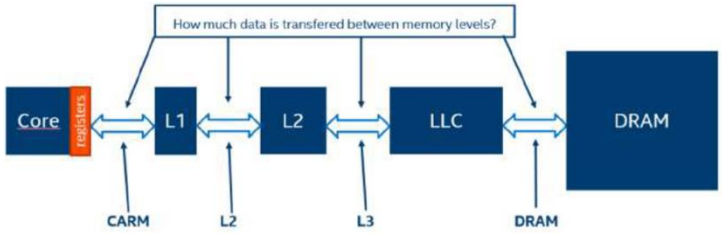


[Stanford CS217]

Hierarchical Roofline

- Real processors have multiple levels of memory
  - Registers
  - L1, L2, L3 cache
  - MCDRAM/HBM (KNL/GPU device memory)
  - DDR (main memory)
  - NVRAM (non-volatile memory)
- Applications can have locality in each level
  - Unique data movements imply unique AI's
  - Moreover, each level will have a unique bandwidth

Modern architectures are complicated!

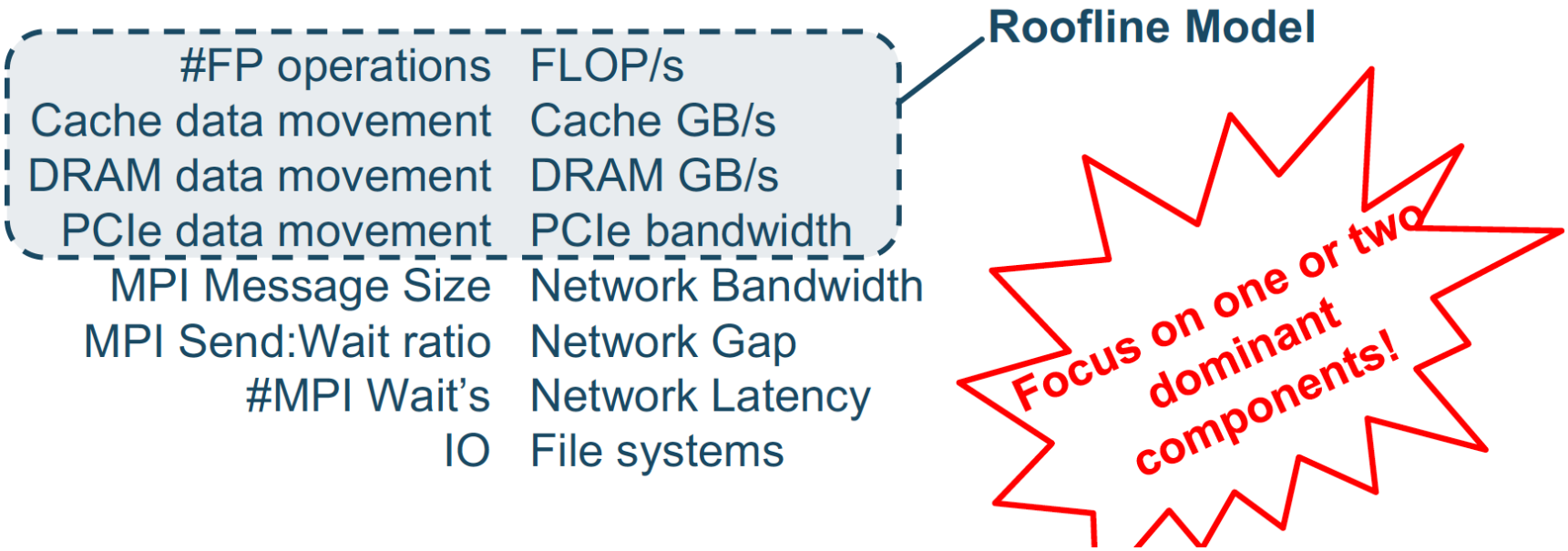


Intel Haswell CPU[1]

[Stanford CS217]

Hierarchical Roofline

- **Many components contribute to the kernel run time**
- **An interplay of application characteristics and machine characteristics**

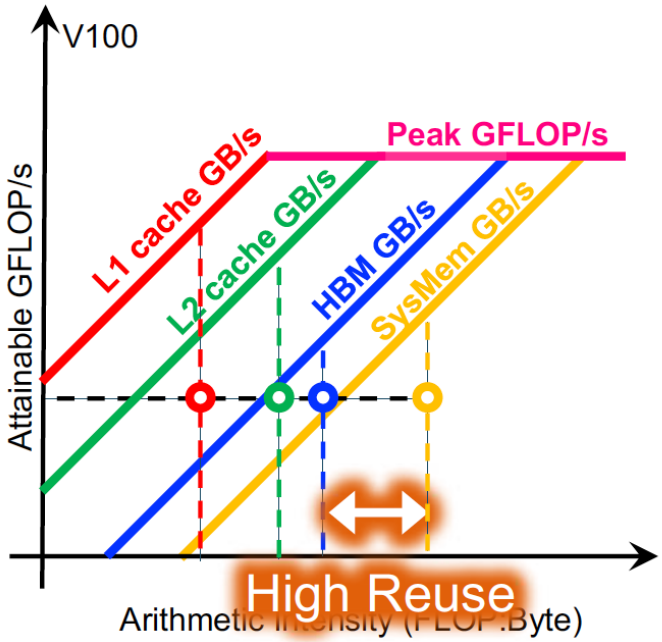


[NERSC]

Hierarchical Roofline

- **Superposition of multiple Rooflines**
  - **Incorporate full memory hierarchy**
  - **Arithmetic Intensity =**

    FLOPs / Bytes$_{L1/L2/HBM/SysMem}$



- **Each kernel will have multiple AI's**

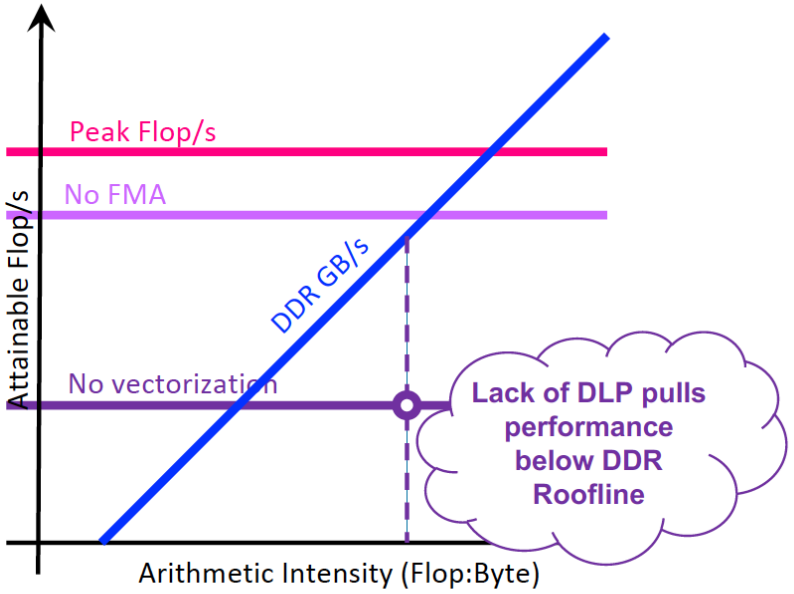
  but one observed GFLOP/s performance

- **Hierarchical Roofline tells you about cache locality**

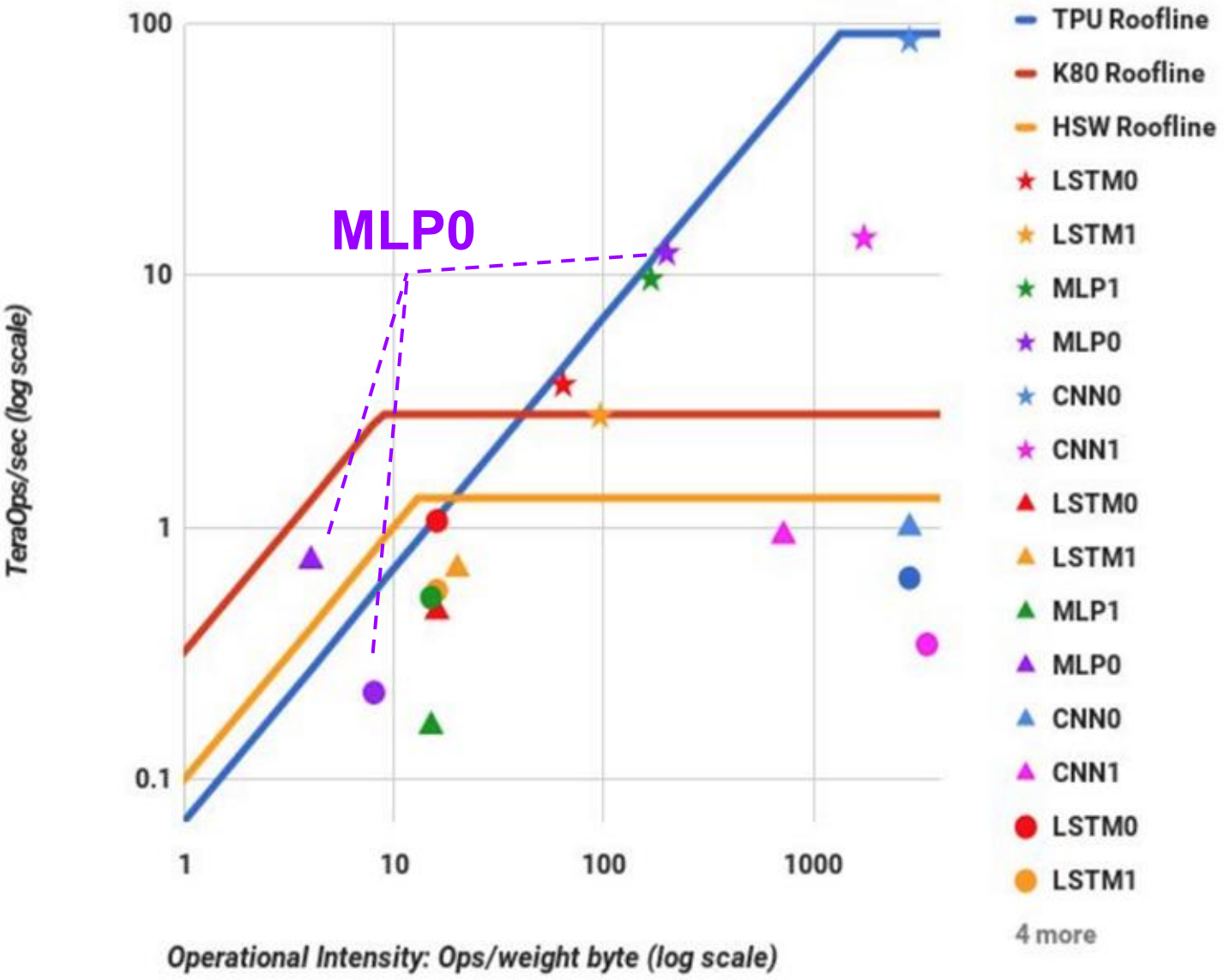
[NERSC]

Data, Instruction, Thread-level Parallelism

- We have assumed one can attain peak flops
- In reality, this is premised on sufficient…
  - Use special instructions (e.g. fused multiply-add)
  - Vectorization/SIMD (16 flops per instruction)
  - unrolling, out-of-order execution (hide FPU latency)
  - OpenMP across multiple cores
- Without these, …
  - Peak performance is not attainable
  - Some kernels can transition from memory-bound to compute-bound
  - n.b. in reality, DRAM bandwidth is often tied to DLP and TLP (single core can't saturate BW w/scalar code)

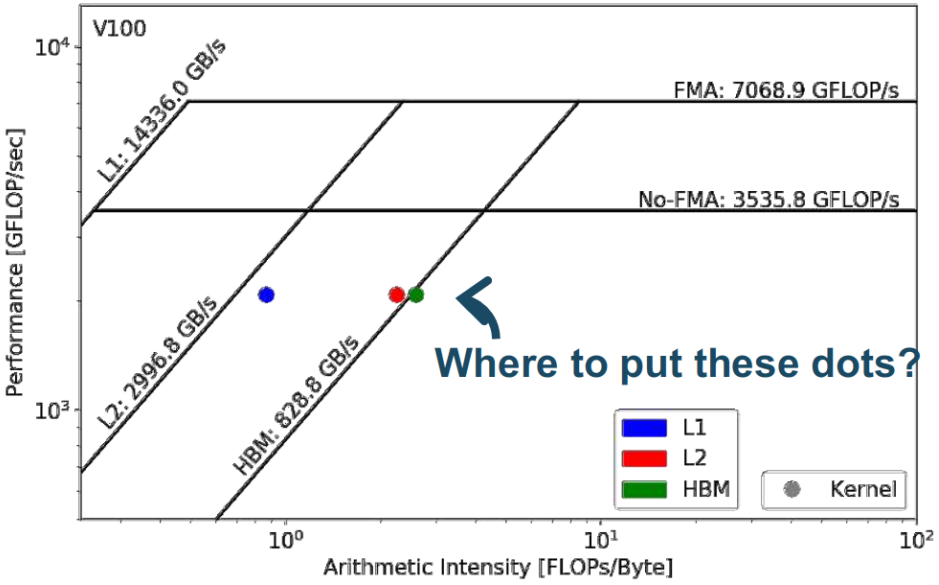


How to get Google TPUv1 Roofline Data



Source: Google

Tools to Construct Roofline



Require three raw measurements:

- **Runtime**
- **FLOPs**
- **Bytes (on each cache level)**

In order to calculate AI and GFLOP/s:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Data Movement}}$$

(FLOPs/Byte)

$$\text{Performance} = \frac{\text{FLOPs}}{\text{Runtime}}$$

(GFLOP/s)

Tools to Construct Roofline

1. **Collect Roofline ceilings**
   - **compute** (FMA/no FMA) and **bandwidth** (DRAM, L2, …)
   - ERT: https://bitbucket.org/berkeleylab/cs-roofline-toolkit
2. **Collect application performance**
   - **FLOPs**, **bytes** (DRAM, L2, …), **runtime**
   - SDE, VTune, LIKWID, Advisor, nvprof, …
3. **Plot Roofline with Python Matplotlib (or other tools of your preference)**
   - **arithmetic intensity**, **GFLOP/s** performance, **ceilings**
   - example scripts: https://github.com/cyanguwa/nersc-roofline

Key Design Objectives for DNN Processor

- **Increase Throughput and Reduce Latency**
  - Reduce time per MAC
    - Reduce critical path → increase clock frequency
    - Reduce instruction overhead
  - Avoid unnecessary MACs (save cycles)
  - Increase number of processing elements (PE) → more MACs in parallel
    - Increase area density of PE or area cost of system
  - Increase PE utilization* → keep PEs busy
    - Distribute workload to as many PEs as possible
    - Balance the workload across PEs
    - Sufficient memory bandwidth to deliver workload to PEs (reduce idle cycles)
- Low latency has an additional constraint of **small batch size**

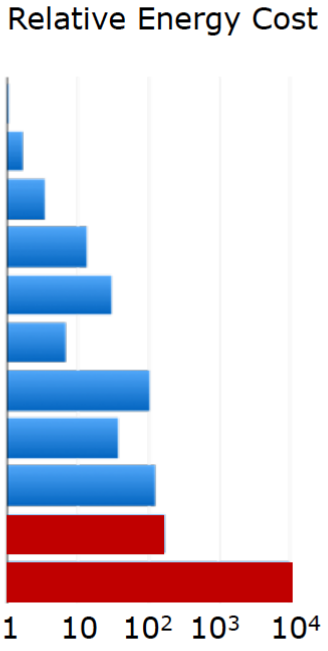
*(100% = peak performance)

Key Design Objectives for DNN Processor

- **Reduce Energy and Power Consumption**
  - Reduce data movement as it dominates energy consumption
    - Exploit data reuse
  - Reduce energy per MAC
    - Reduce switching activity and/or capacitance
    - Reduce instruction overhead
  - Avoid unnecessary MACs
- Power consumption is limited by heat dissipation, which limits the maximum # of MACs in parallel (i.e., throughput)

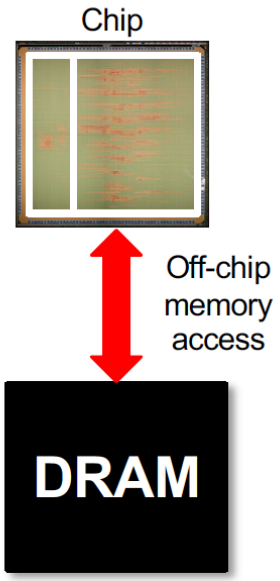
Operation:	Energy (pJ)
8b Add	0.03
16b Add	0.05
32b Add	0.1
16b FP Add	0.4
32b FP Add	0.9
8b Multiply	0.2
32b Multiply	3.1
16b FP Multiply	1.1
32b FP Multiply	3.7
32b SRAM Read (8KB)	5
32b DRAM Read	640

[**Horowitz**, *ISSCC* 2014]



Specifications to Evaluate Metrics

- **Accuracy**
  - Difficulty of dataset and/or task should be considered
  - Difficult tasks typically require more complex DNN models
- **Throughput**
  - Number of PEs with utilization (not just peak performance)
  - Runtime for running specific DNN models
- **Latency**
  - Batch size used in evaluation
- **Energy and Power**
  - Power consumption for running specific DNN models
  - Off-chip memory access (e.g., DRAM)
- **Hardware Cost**
  - On-chip storage, # of PEs, chip area + process technology
- **Flexibility**
  - Report performance across a wide range of DNN models
  - Define range of DNN models that are efficiently supported



Evaluation Process

The evaluation process for whether a DNN system is a viable solution for a given application might go as follows:

1. **Accuracy** determines if it can perform the given task
2. **Latency and throughput** determine if it can run fast enough and in real-time
3. **Energy and power consumption** will primarily dictate the form factor of the device where the processing can operate
4. **Cost**, which is primarily dictated by the chip area and external interfaces, determines how much one would pay for this solution
5. **Flexibility** determines the range of tasks it can support

Example: Metrics of Eyeriss Chip

ASIC Specs	Input
Process Technology	65nm LP TSMC (1.0V)
Total Core Area (mm^2)	12.25
Total On-Chip Memory (kB)	192
Number of Multipliers	168
Clock Frequency (MHz)	200
Core area (mm^2) /multiplier	0.073
On-Chip memory (kB) / multiplier	1.14
Measured or Simulated	Measured

Metric	Units	Input
Name of CNN Model	Text	AlexNet
Top-5 error classification on ImageNet	#	19.8
Supported Layers		All CONV
Bits per weight	#	16
Bits per input activation	#	16
Batch Size	#	4
Runtime	ms	115.3
Power	mW	278
Off-chip Access per Image Inference	MBytes	3.85
Number of Images Tested	#	100

Comprehensive Coverage for Evaluation

- **All metrics** should be reported for fair evaluation of design tradeoffs
- Examples of what can happen if certain metric is omitted:
  - **Without the accuracy given for a specific dataset and task**, one could run a simple DNN and claim low power, high throughput, and low cost – however, the processor might not be usable for a meaningful task
  - **Without reporting the off-chip bandwidth**, one could build a processor with only multipliers and claim low cost, high throughput, high accuracy, and low chip power – however, when evaluating system power, the off-chip memory access would be substantial
- Are results measured or simulated? On what test data?
- Hardware should be evaluated on a wide range of DNNs
  - No guarantee that DNN algorithm designer will use a given DNN model or given reduce complexity approach. **Need flexible hardware!**

MLPerf: Workloads for Benchmarking

23 Companies
7 Institutions



First results in Dec 2018

https://mlperf.org/

- A broad suite of DNN models to serve as a common set of benchmarks to measure the performance and enable fair comparison of various software frameworks, hardware accelerators, and cloud platforms for both training and inference of DNNs. **(edge compute in the works!)**
- The suite includes a wide range of DNNs (e.g., CNN, RNN, etc.) for a variety of tasks include image classification, object identification, translation, speech-to-text, recommendation, sentiment analysis and reinforcement learning.
- Categories: cloud/edge; training/inference; closed/open

Specifications of DNN Models

- **Accuracy**
  - Define task and dataset
- **Shape of DNN Model** ("Network Architecture")
  - # of layers, filter size (R, S), # of channels (C), # of filters (M)
- **# of Weights & Activations (storage capacity)**
  - Number of non-zero (NZ) weights and activations
- **# of MACs (operations)**
  - Number of non-zero (NZ) MACS

Specifications of DNN Models

Metrics	AlexNet	VGG-16	GoogLeNet (v1)	ResNet-50
Accuracy (top-5 error)*	19.8	8.80	10.7	7.02
# of CONV Layers	**5**	**16**	**21**	**49**
# of Weights	2.3M	14.7M	6.0M	23.5M
# of MACs	666M	15.3G	1.43G	3.86G
# of NZ MACs**	**394M**	**7.3G**	**806M**	**1.5G**
# of FC layers	**3**	**3**	**1**	**1**
# of Weights	58.6M	124M	1M	2M
# of MACs	58.6M	124M	1M	2M
# of NZ MACs**	**14.4M**	**17.7M**	**639k**	**1.8M**
Total Weights	**61M**	**138M**	**7M**	**25.5M**
Total MACs	**724M**	**15.5G**	**1.43G**	**3.9G**
# of NZ MACs**	**409M**	**7.3G**	**806M**	**1.5G**

*Single crop results: https://github.com/jcjohnson/cnn-benchmarks

**# of NZ MACs computed based on 50,000 validation images

Weights & MACs -> Energy & Latency

- ***Warning:*** *Fewer weights and MACs (indirect metrics) do not necessarily result in lower energy consumption or latency (direct metrics). Other factors also important such as filter shape, batch size and hardware mapping.*

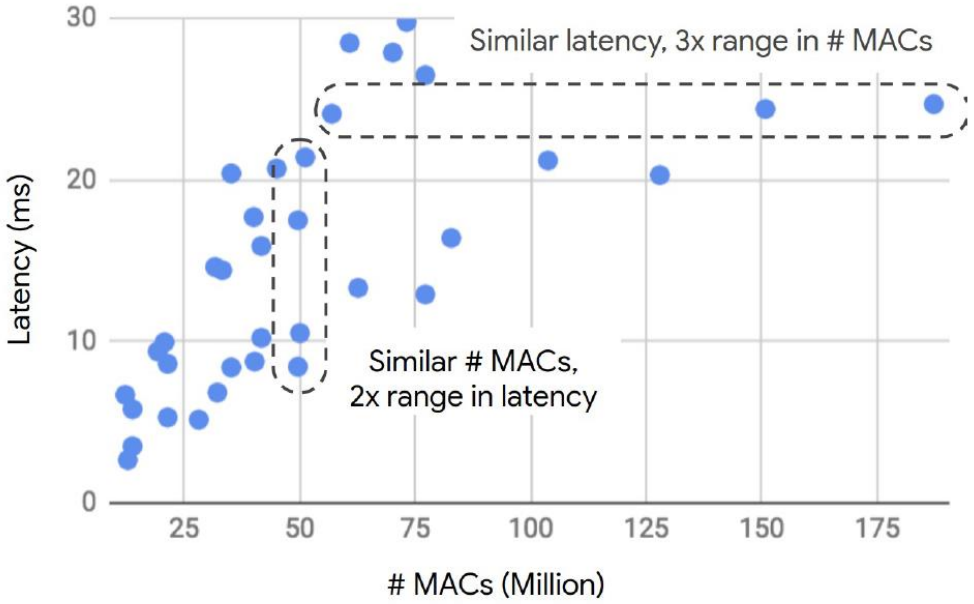
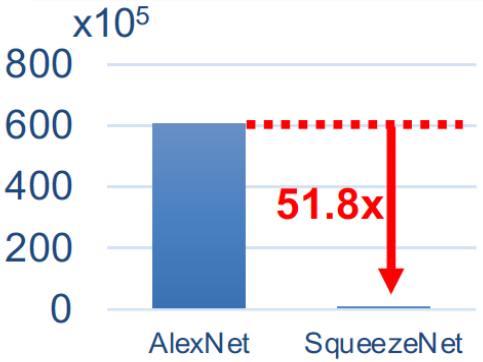


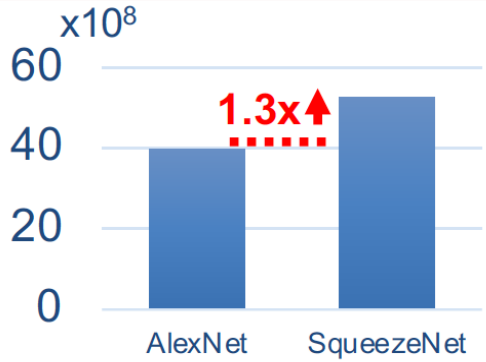
Image Source:
Google AI Blog

[**Yang**, *CVPR* 2017], [**Chen**, *SysML*, 2018], [**Yang**, *ECCV* 2018]

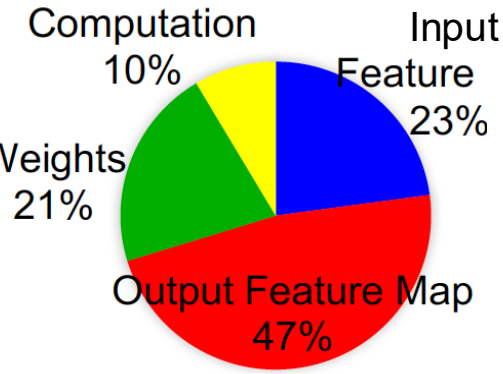
Example: AlexNet Vs. SqueezeNet



of Weights



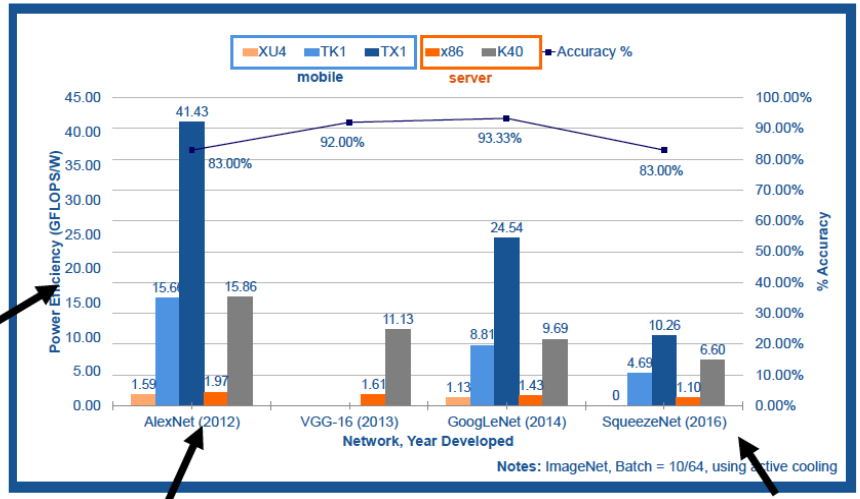
Normalized Energy



Energy Breakdown (SqueezeNet)

Results for SqueezeNetv1.0
Batch size=48

Power Efficiency (GFLOPS/W)



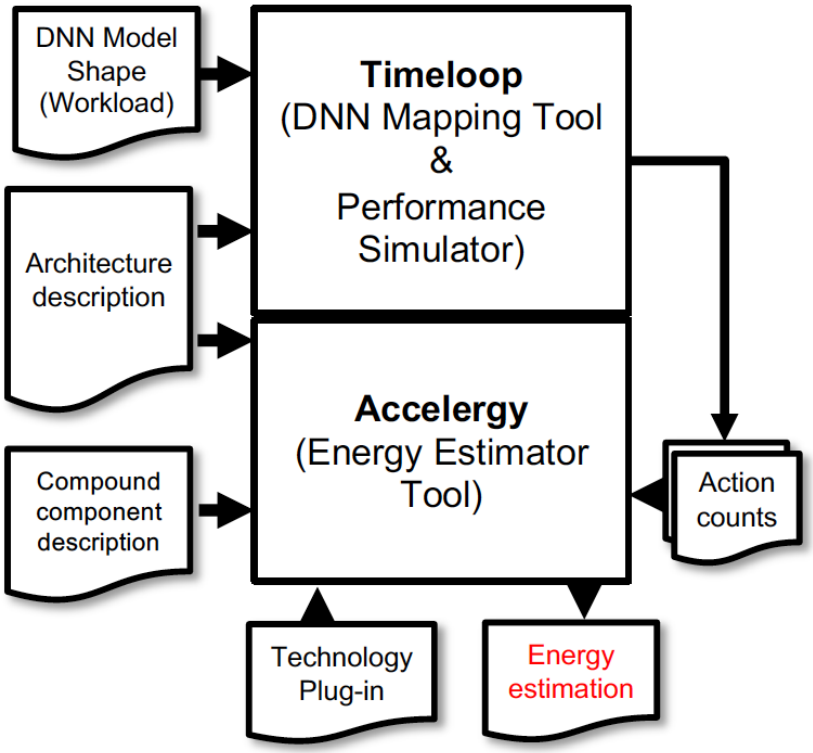
AlexNet

SqueezeNet

[**Movidius**, *Hot Chips* 2016]

DNN Processor Evaluation Tools

- **Require systematic way to**
  - Evaluate and compare wide range of DNN processor designs
  - Rapidly explore design space
- **Accelergy [Wu, *ICCAD* 2019]**
  - Early-stage energy estimation tool at the architecture level
    - Estimate energy consumption based on architecture level components (e.g., # of PEs, memory size, on-chip network)
  - Evaluate architecture level energy impact of emerging devices
    - Plug-ins for different technologies
- **Timeloop [Parashar, *ISPASS* 2019]**
  - DNN mapping tool
  - Performance Simulator → Action counts

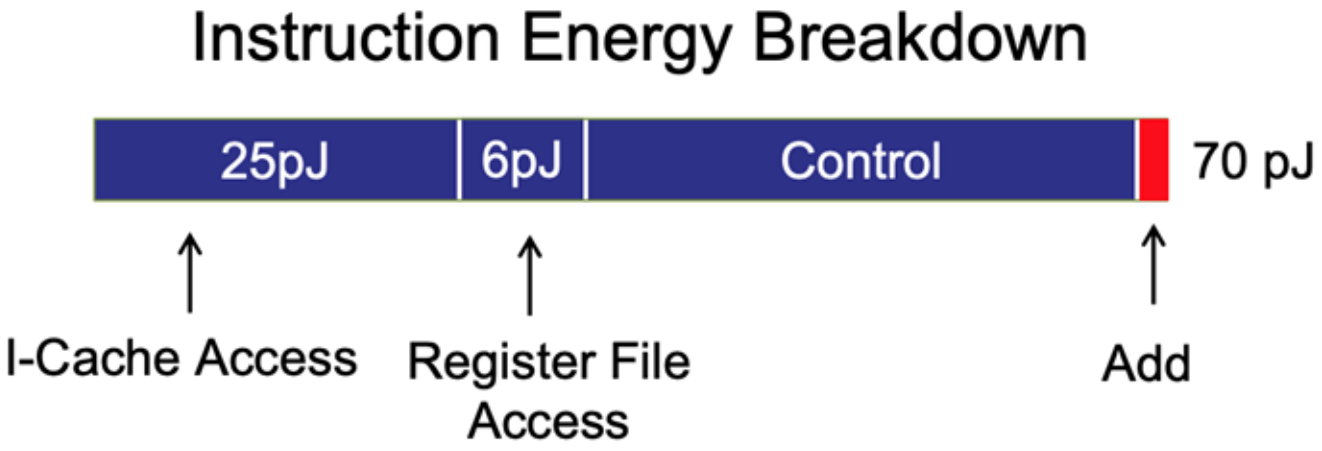


Labs and final project

Open-source code available at:
http://accelergy.mit.edu

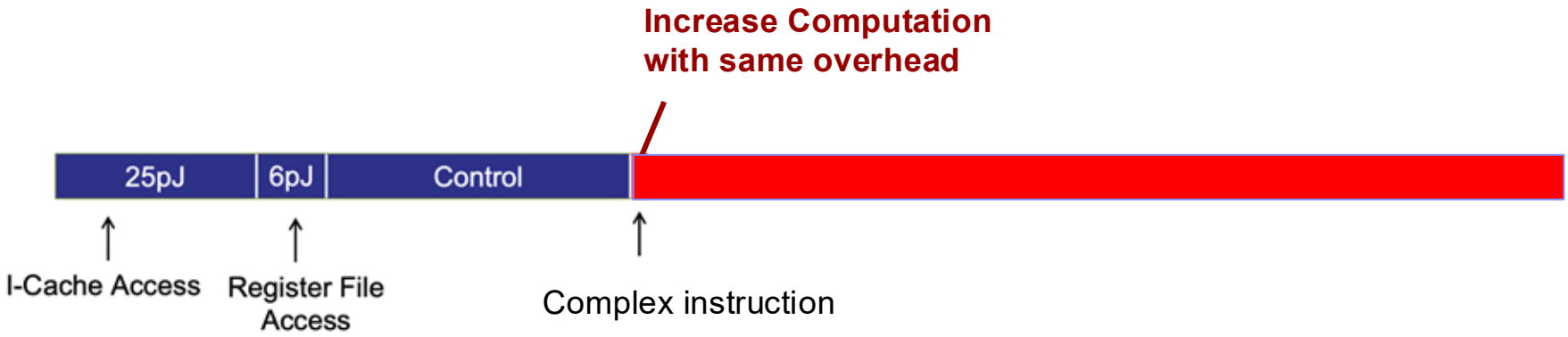
What about Energy?

- Energy breakdown of an add instruction in a 45nm CPU
- How can we optimize this?



Source: Mark Horowitz "Computing's Energy Problem (and What we can do about it)" ISSCC 2014

To Make it Better: Amortizing Overhead



Half-precision Fused Multiply-Add

4-way dot-product

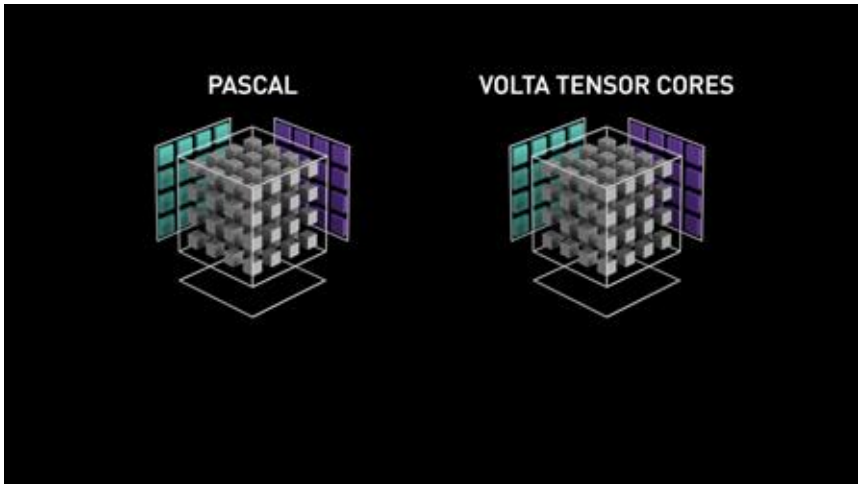
16x16 matrix multiplication

Operation	Energy**	Overhead*
HFMA	1.5pJ	2000%
HDP4A	6.0pJ	500%
HMMA	110pJ	27%

Source: Bill Dally "Hardware for Deep Learning" SysML 2018

Real Chip Examples

GPU

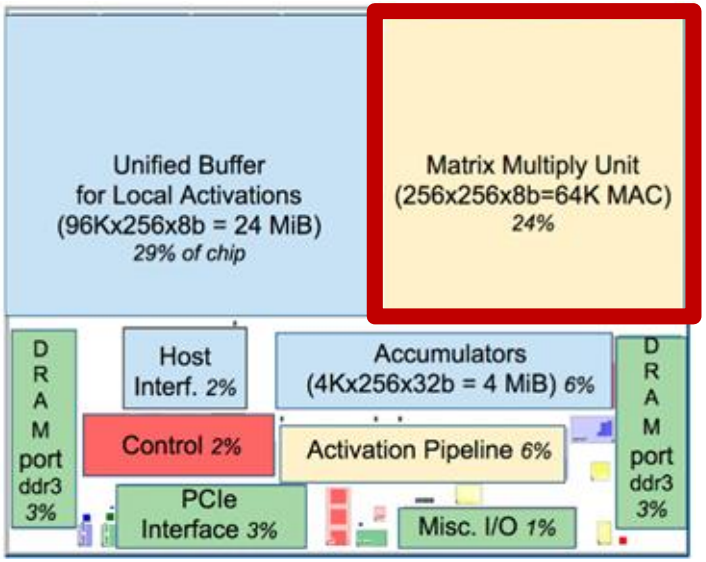


16x16 = 256* MAC/cycle

*~ 500 tensor cores per GPU

Source: Nvidia

ASIC (TPUv1)

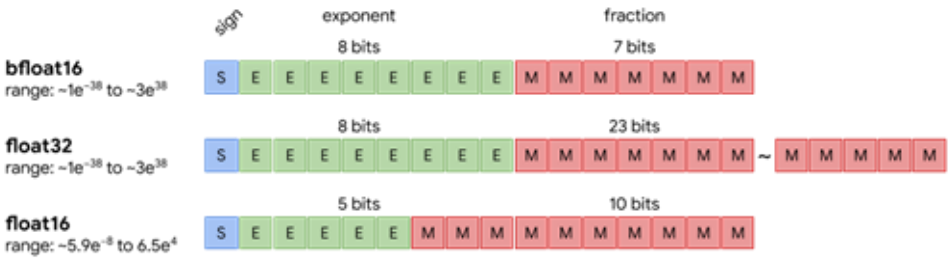


256x256 = 64 kMAC/cycle

Source: Google

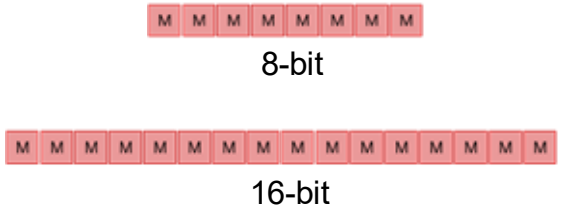
Numerical Format and Precision

Floating Point



- IEEE standard includes FP32 and FP16
- Many exotic FP numbers in DNN
  - E.g. bfloat, minifloat

Integer



- Whole numbers only
- (typically) much cheaper circuit area and power

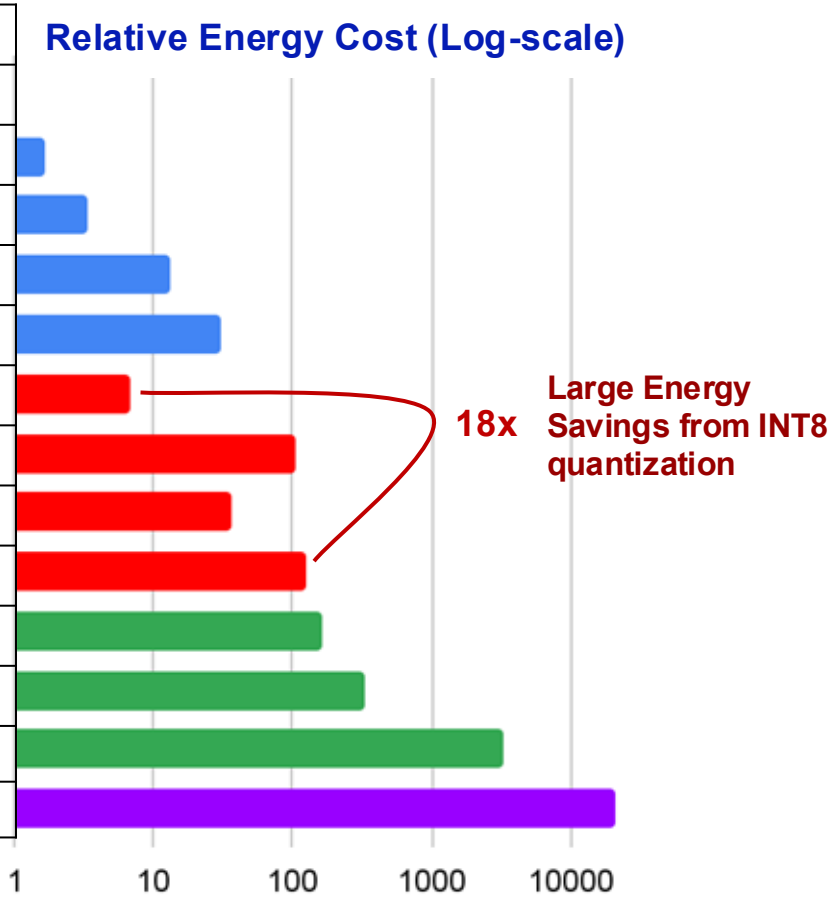
Source: Google

Cost of Arithmetic Operations

Operation	Precision	Energy (pJ)
Addition	INT8	0.03
	INT16	0.05
	INT32	0.1
	FP16	0.4
	FP32	0.9
Multiplication	INT8	0.2
	INT32	3.1
	FP16	1.1
	FP32	3.7
32-bit SRAM Read (8 kB)		5
32-bit SRAM Read (32 kB)		10
32-bit SRAM Read (1 MB)		100
32-bit DRAM Read		640

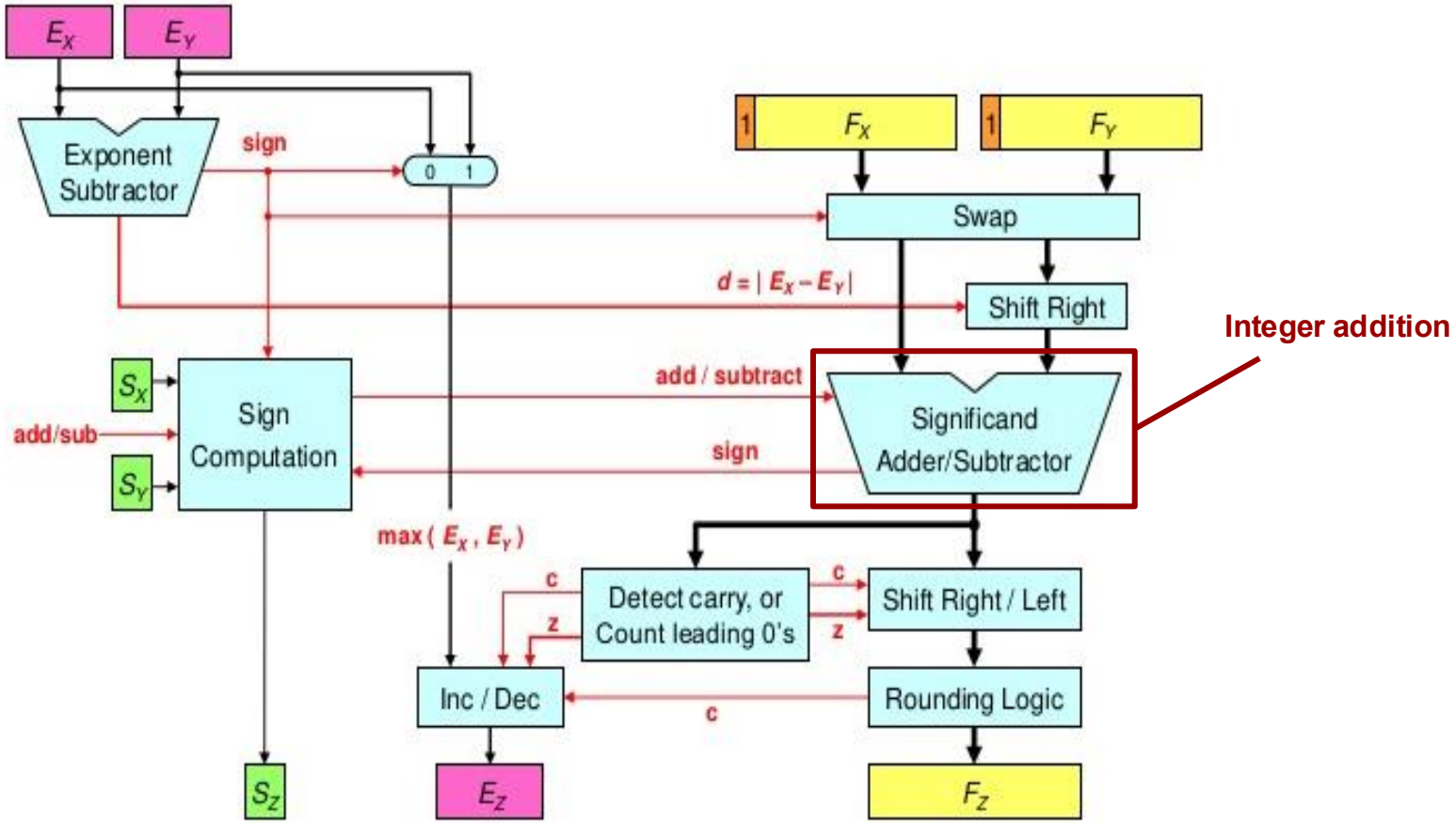
4x

16x



Adapted from Mark Horowitz "Computing's Energy Problem (and What we can do about it)" ISSCC 2014

Complexity of Floating-Point Addition

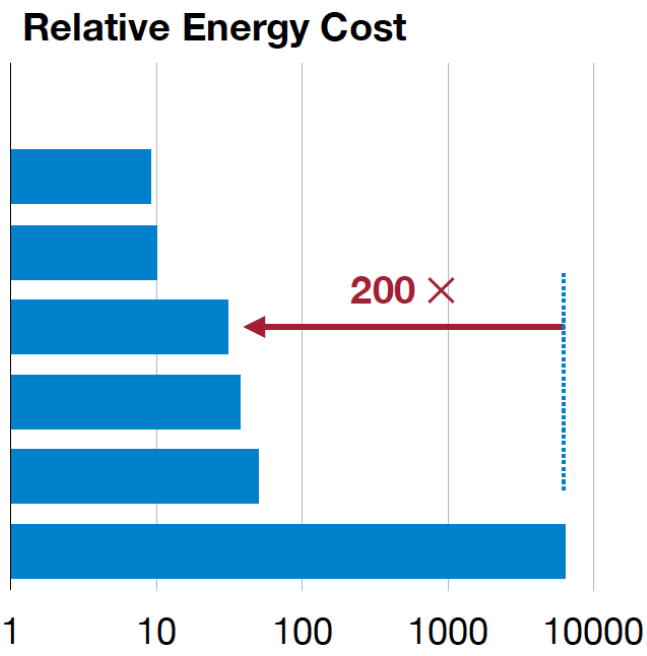


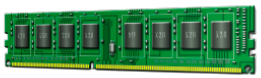
Energy Consumption

Data movement → more memory reference → more energy

Operation	Energy [pJ]
32 bit int ADD	0.1
32 bit float ADD	0.9
32 bit Register File	1
32 bit int MULT	3.1
32 bit float MULT	3.7
32 bit SRAM Cache	5
32 bit DRAM Memory	640

Rough Energy Cost For Various Operations in 45nm 0.9V



1  = 200 ×+

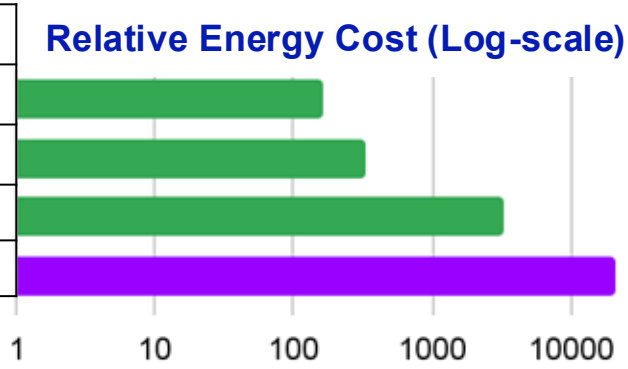
This image is in the public domain

Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

Memory Hierarchy Optimizations

1. **Get data close to the computation. (LOCALITY)**
2. **Once data is close - perform all computations with this data. (REUSE)**

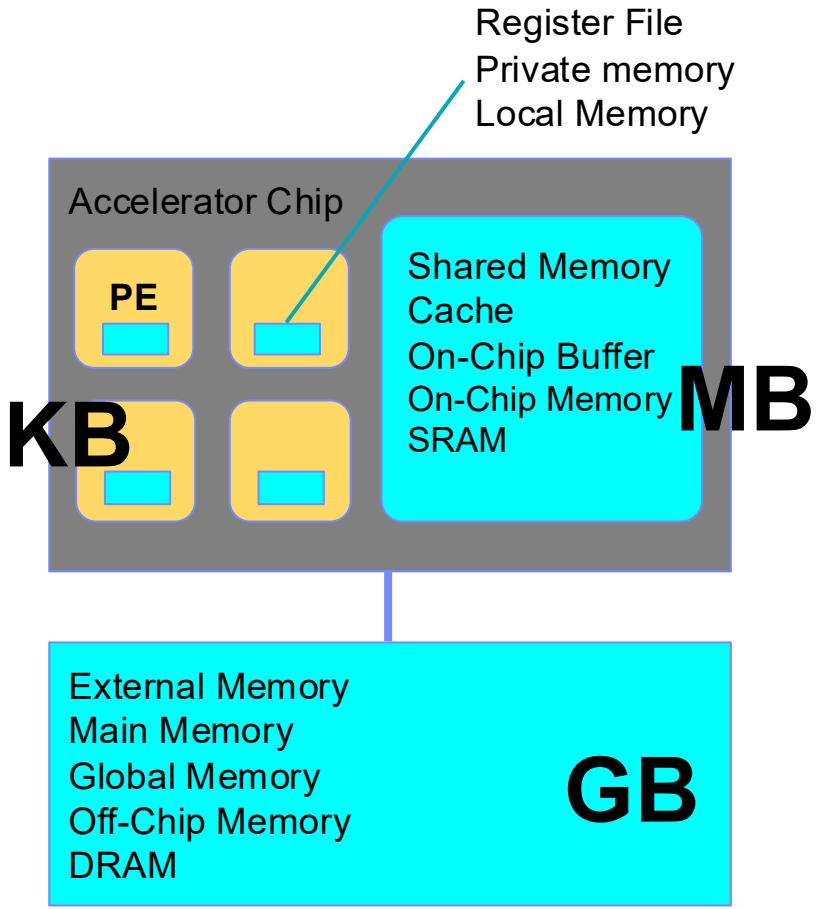
Operation	Energy (pJ)
32-bit SRAM Read (8 kB)	5
32-bit SRAM Read (32 kB)	10
32-bit SRAM Read (1 MB)	100
32-bit DRAM Read	640



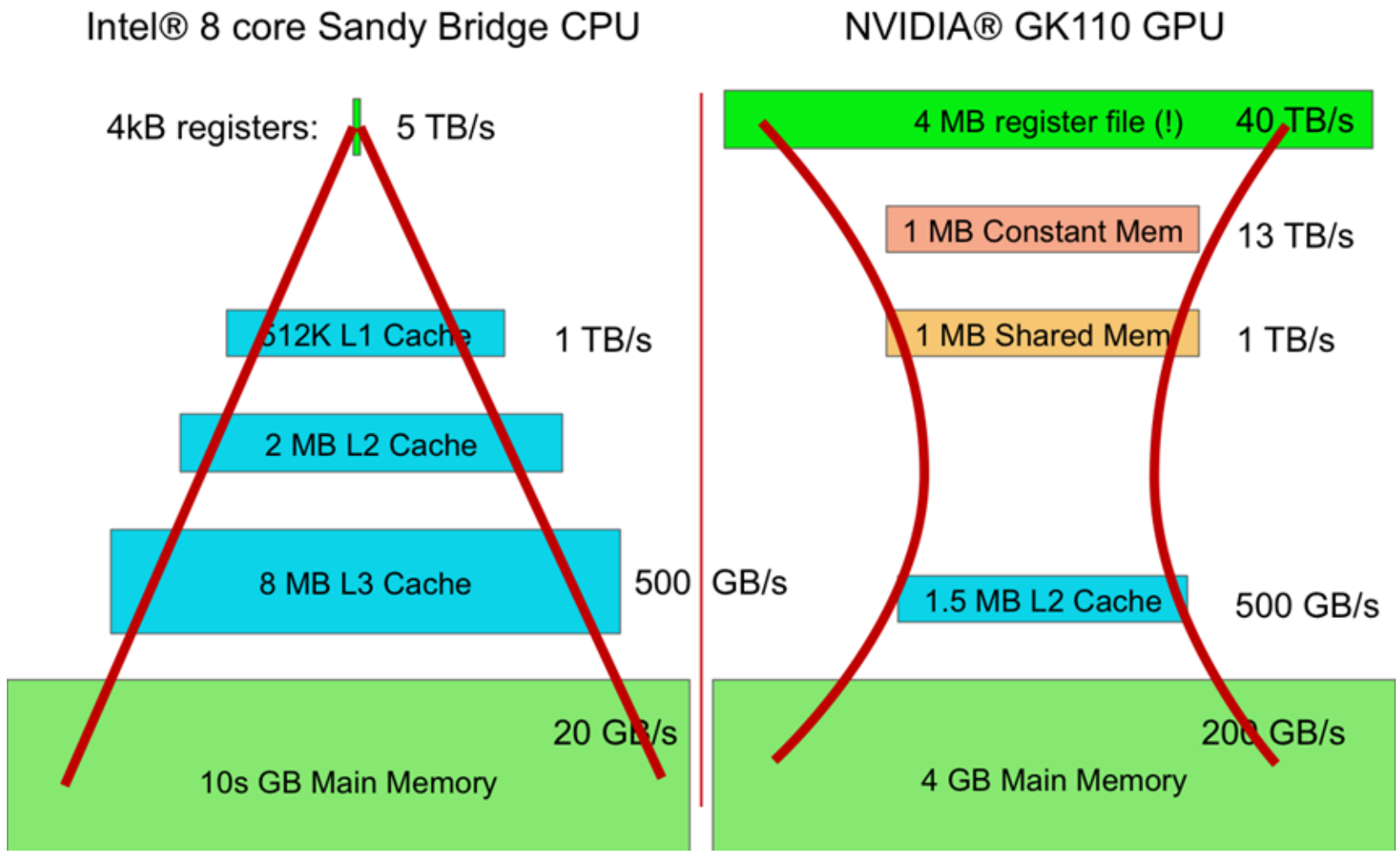
Memory Hierarchy

Why do we have a memory hierarchy?

- The closer you get to compute, the more $$ and scarce the memory resource becomes
- In *most* cases, the DNN parameters live off chip and are fetched layer-by-layer or tile-by-tile
- Data locality: how to get data close to the PEs (to keep them fully utilized)



Memory Hierarchy Example



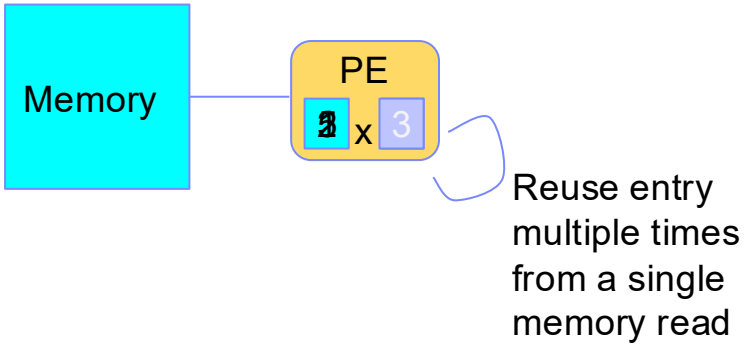
Source: Nvidia

Data Reuse

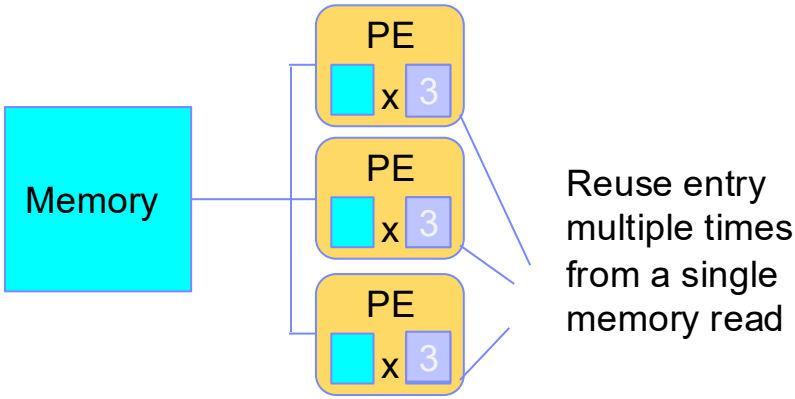
Temporal Reuse

Spatial Reuse

Read once from memory, use same data multiple times by same PE



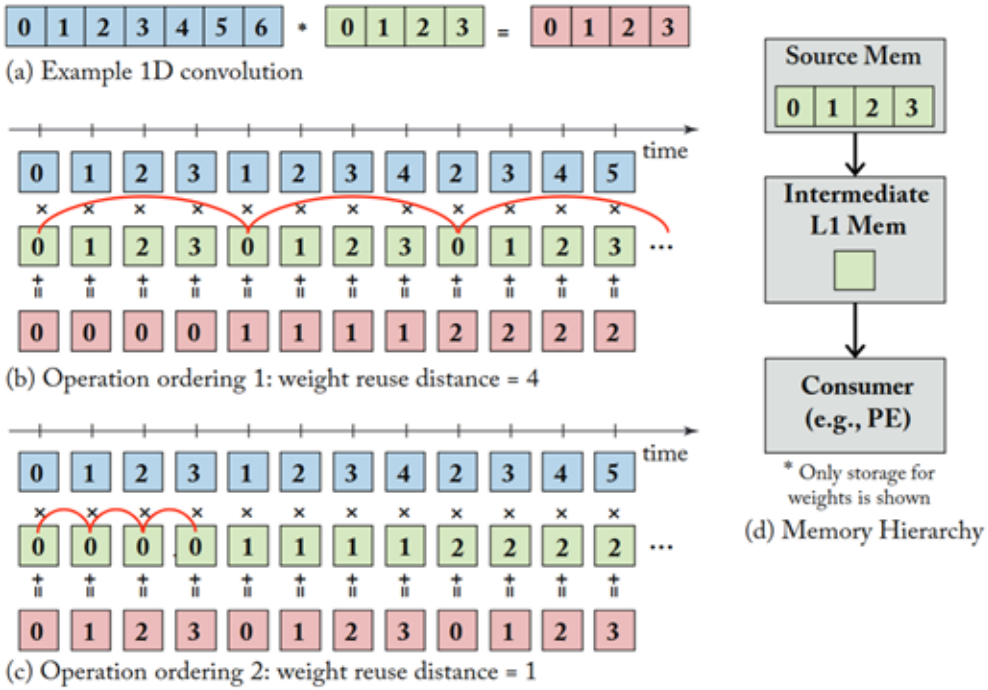
Read once from memory, use same data multiple times by multiple PEs



Temporal Reuse Example

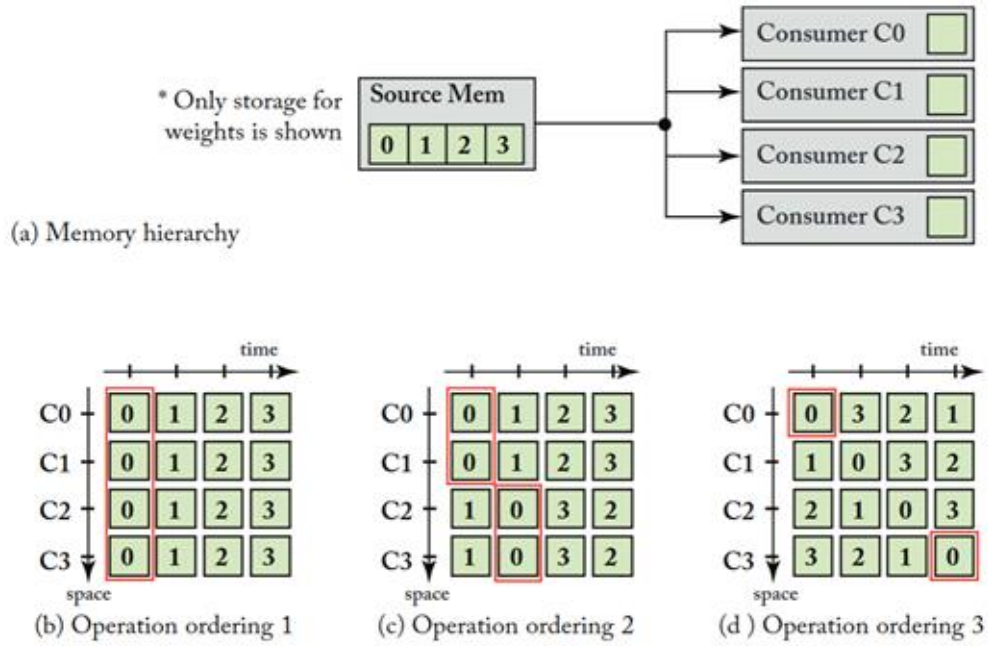
Reuse Distance

- Number of memory accesses
- Size of your on-chip memory
- Smaller is better



Source: V. Sze et al. "Efficient Processing of Deep Neural Networks" 2020

Spatial Reuse Example



Why a smaller reuse distance with spatial reuse?

- Fewer memory read ports
  - Less area
- Fewer reads
  - Less power

Source: V. Sze et al. "Efficient Processing of Deep Neural Networks" 2020